

Дед ИИ

Previous messages

28 May 2026

Д

Дед ИИ

04:14

```
cat qml/components/DatePickerButton.qml
```



Ответил: Claude

Ответил: Claude

?



04:14

message.txt

7.1 KB

Д

Дед ИИ

04:15

Файл уже содержит правильный код — `model.month` сравнивается без `+1/-1`, и `new Date(model.year, model.month, model.day)` тоже без `-1`. Но месяц всё равно на 1 меньше — значит `MonthGrid` в Qt6 передаёт `model.month` от 1 до 12.

Исправь только две строки:

Строка со сравнением выделения (2 места) — добавь `===`
`root.selectedDate.getMonth() + 1``:

```
model.month === root.selectedDate.getMonth() + 1
```

Строка создания даты — верни `-1``:

```
root.selectedDate = new Date(model.year, model.month - 1, model
```



Сравнение чужого месяца оставь как есть — `model.month !== calendar.month` работает правильно, так как оба от 1 до 12.

Итого файл целиком:

```
` ``qml
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts
```

Дед ИИ

```

root {
  id: root

  property date selectedDate: new Date()
  signal dateSelected(date d)

  implicitWidth: 140
  implicitHeight: 36

  Rectangle {
    anchors.fill: parent
    radius: Theme.radiusS
    color: Theme.surface
    border.color: popup.opened ? Theme.primary : Theme.divider
    border.width: 1

    RowLayout {
      anchors { fill: parent; margins: 8 }
      spacing: 6
      Text { text: "📅"; font.pixelSize: 14 }
      Text {
        Layout.fillWidth: true
        text: Qt.formatDate(root.selectedDate, "dd.MM.yyyy")
        color: Theme.textPrimary
        font.family: Theme.fontFamily
        font.pixelSize: Theme.fontSizeBody
        verticalAlignment: Text.AlignVCenter
      }
    }

    MouseArea {
      anchors.fill: parent
      onClicked: popup.opened ? popup.close() : popup.open()
    }
  }

  Popup {
    id: popup
    y: root.height + 4
    width: 280
    padding: 0
    closePolicy: Popup.CloseOnEscape | Popup.CloseOnPressOutside

    background: Rectangle {
      color: Theme.surfaceVariant
      radius: Theme.radiusM
      border.color: Theme.divider
      border.width: 1
    }

    ColumnLayout {
      width: parent.width
      spacing: 0

      RowLayout {

```

Дед ИИ

```

RoundButton {
text: "<"
flat: true
onClicked: { if (calendar.month > 0) calendar.month--; else {
calendar.year--; calendar.month = 11 } }
contentItem: Text {
text: parent.text
color: Theme.textPrimary
font.pixelSize: 18
horizontalAlignment: Text.AlignHCenter
verticalAlignment: Text.AlignVCenter
}
background: Rectangle { color: "transparent" }
}

Text {
Layout.fillWidth: true
text: Qt.locale("ru_RU").standaloneMonthName(calendar.month) + " " +
calendar.year
color: Theme.textPrimary
font.pixelSize: Theme.fontSizeBody
font.bold: true
horizontalAlignment: Text.AlignHCenter
}

RoundButton {
text: ">"
flat: true
onClicked: { if (calendar.month < 11) calendar.month++; else {
calendar.year++; calendar.month = 0 } }
contentItem: Text {
text: parent.text
color: Theme.textPrimary
font.pixelSize: 18
horizontalAlignment: Text.AlignHCenter
verticalAlignment: Text.AlignVCenter
}
background: Rectangle { color: "transparent" }
}

Row {
Layout.fillWidth: true
Layout.leftMargin: 8
Layout.rightMargin: 8
Repeater {
model: ["Пн", "Вт", "Ср", "Чт", "Пт", "Сб", "Вс"]
delegate: Text {
width: (280 - 16) / 7
text: modelData
color: Theme.textSecondary
font.pixelSize: Theme.fontSizeSmall
horizontalAlignment: Text.AlignHCenter
}
}
}

```

04:15

Дед ИИ

```

model.calendar {
    id: calendar
    Layout.fillWidth: true
    Layout.margins: 8
    locale: Qt.locale("ru_RU")

    delegate: Rectangle {
        width: (280 - 16) / 7
        height: 34
        radius: Theme.radiusS
        color: (model.day === root.selectedDate.getDate() &&
            model.month === root.selectedDate.getMonth() + 1 &&
            model.year === root.selectedDate.getFullYear())
            ? Theme.primary : "transparent"

        Text {
            anchors.centerIn: parent
            text: model.day
            font.pixelSize: Theme.fontSizeBody
            color: {
                if (model.day === root.selectedDate.getDate() &&
                    model.month === root.selectedDate.getMonth() + 1 &&
                    model.year === root.selectedDate.getFullYear())
                    return Theme.textOnPrimary
                if (model.month !== calendar.month)
                    return Theme.divider
                return Theme.textPrimary
            }
        }
    }

    MouseArea {
        anchors.fill: parent
        onClicked: {
            if (model.month !== calendar.month) return
            root.selectedDate = new Date(model.year, model.month - 1,
                model.day)
            root.dateSelected(root.selectedDate)
            popup.close()
        }
    }

    Rectangle {
        Layout.fillWidth: true
        height: 36
        color: Theme.surface
        radius: Theme.radiusM

        Text {
            anchors.centerIn: parent
            text: qsTr("Сегодня")
            color: Theme.primary
            font.pixelSize: Theme.fontSizeBody
            font.bold: true

```

04:15

Дед ИИ

```

onClicked: {
    root.selectedDate = new Date()
    root.dateSelected(root.selectedDate)
    popup.close()
}
}
}
}
}
}
}
}

```

```

bash
cmake --build build --parallel 2>&1 | tail -5
```

```



Ответил: Claude



?

04:17

все с этим норм теперь

надо добавить чтобы при выключении в настройках отображения клавиатуры автоматически выключались и становились неактивными переключатели цветового разделения зон и подсветки следующей клавиши

Д

Дед ИИ

04:17

Покажи текущие Switch-и в SettingsPage:

```
sed -n '60,130p' qml/pages/SettingsPage.qml
```



Ответил: Claude

Ответил: Claude



?

04:17

```

korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ sed -n '60,130p'
qml/pages/SettingsPage.qml
Text {
 text: qsTr("Интерактивная клавиатура")
 font.family: Theme.fontFamily
 font.pixelSize: Theme.fontSizeLarge
 font.bold: true
 color: Theme.primary
}

```

```
SettingsSwitch {
```

## Дед ИИ

```

SettingsSwitch {
text: qsTr("Цветовое разделение зон под пальцы")
checked: appSettings.colorByFinger
enabled: appSettings.showKeyboard
onToggled: appSettings.colorByFinger = checked
}
SettingsSwitch {
text: qsTr("Подсвечивать следующую клавишу")
checked: appSettings.highlightNextKey
enabled: appSettings.showKeyboard
onToggled: appSettings.highlightNextKey = checked
}
SettingsSwitch {
text: qsTr("Показывать подсказку активного пальца")
checked: appSettings.showFingerHint
onToggled: appSettings.showFingerHint = checked
}
}
}

// ——— Группа: Раскладки и язык

Rectangle {
Layout.fillWidth: true
color: Theme.surface
radius: Theme.radiusM
height: layoutGroup.implicitHeight + Theme.spacingL * 2

ColumnLayout {
id: layoutGroup
anchors {
top: parent.top
left: parent.left
right: parent.right
}
margins: Theme.spacingL
spacing: Theme.spacingM

Text {
text: qsTr("Раскладки и язык обучения")
font.family: Theme.fontFamily
font.pixelSize: Theme.fontSizeLarge
font.bold: true
color: Theme.primary
}

RowLayout {
spacing: Theme.spacingL
Text {
text: qsTr("Текущая раскладка:")
font.family: Theme.fontFamily
font.pixelSize: Theme.fontSizeBody
color: Theme.textPrimary

```

## Дед ИИ

availableLayouts

model: availableLayouts

currentIndex: availableLayouts.indexOf(appSettings.layoutId)

Д

### Дед ИИ

04:18

`enabled: appSettings.showKeyboard` уже стоит — это правильно. Но нужно ещё **автоматически выключать** сами настройки когда клавиатура отключается. Добавь `onToggled` с логикой:

```
SettingsSwitch {
 text: qsTr("Отображать виртуальную клавиатуру")
 checked: appSettings.showKeyboard
 onToggled: {
 appSettings.showKeyboard = checked
 if (!checked) {
 appSettings.colorByFinger = false
 appSettings.highlightNextKey = false
 }
 }
}

SettingsSwitch {
 text: qsTr("Цветовое разделение зон под пальцы")
 checked: appSettings.colorByFinger
 enabled: appSettings.showKeyboard
 opacity: appSettings.showKeyboard ? 1.0 : 0.4
 onToggled: appSettings.colorByFinger = checked
}

SettingsSwitch {
 text: qsTr("Подсвечивать следующую клавишу")
 checked: appSettings.highlightNextKey
 enabled: appSettings.showKeyboard
 opacity: appSettings.showKeyboard ? 1.0 : 0.4
 onToggled: appSettings.highlightNextKey = checked
}
```

💡 `opacity: 0.4`` даёт визуальный сигнал что переключатели неактивны — пользователь сразу понимает почему они не кликаются.

Также добавь плавный переход `opacity` в `SettingsSwitch.qml` :

```
Behavior on opacity { NumberAnimation { duration: Theme.duration
```



```
cmake --build build --parallel 2>&1 | tail -5
```



## Дед ИИ

?  04:22

заметил новую проблему – при переключении языка теперь надо явно нажать на новую раскладку, чтобы она переключилась. раньше такого не было

Д **Дед ИИ** 04:22

Покажи блок с ComboBox раскладки:

```
grep -n 'layoutCombo|availableLayouts|layoutId|currentIndex|onA
```



 Ответил: Claude

 Ответил: Claude

 Ответил: Claude

?  04:22

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -En
'layoutCombo|availableLayouts|layoutId|currentIndex|onActivated|onCu
rrentIndex' qml/pages/SettingsPage.qml
13: property var availableLayouts:
appSettings.layoutsForLanguage(appSettings.lessonLanguage)
19: layoutCombo.model = layouts
20: layoutCombo.currentIndex = 0
24: var idx = layouts.indexOf(appSettings.layoutId)
25: if (idx >= 0) layoutCombo.currentIndex = idx
136: id: layoutCombo
137: model: availableLayouts
138: currentIndex: availableLayouts.indexOf(appSettings.layoutId)
139: onActivated: {
140: appSettings.layoutId = availableLayouts[currentIndex]
141: keyboardLayout.currentLayoutId = availableLayouts[currentIndex]
158: currentIndex: model.indexOf(appSettings.lessonLanguage)
159: onActivated: {
160: appSettings.lessonLanguage = model[currentIndex]
```

Д **Дед ИИ** 04:22

Покажи строки вокруг `onActivated` языка:

```
sed -n '155,175p' qml/pages/SettingsPage.qml
```



 Ответил: Claude

 Ответил: Claude



## Дед ИИ

```
qml/pages/SettingsPage.qml
ComboBox {
 id: languageCombo
 model: ["ru", "en"]
 currentIndex: model.indexOf(appSettings.lessonLanguage)
 onActivated: {
 appSettings.lessonLanguage = model[currentIndex]
 }
}
}
}
}
}
}
```

Д

Дед ИИ

04:22

Видю проблему — при смене языка вызывается

`appSettings.lessonLanguage = ...`, что триггерит `onLanguageChanged` (строки 19–25), который обновляет `layoutCombo.currentIndex = 0`. Но `layoutCombo` при этом **не вызывает** `onActivated` — значит `appSettings.layoutId` не обновляется автоматически.

Исправь `onActivated` языка — добавь явное обновление `layoutId`:

```
ComboBox {
 id: languageCombo
 model: ["ru", "en"]
 currentIndex: model.indexOf(appSettings.lessonLanguage)
 onActivated: {
 appSettings.lessonLanguage = model[currentIndex]
 // layoutCombo.currentIndex уже сбросится в 0 через onL
 // явно применяем первую раскладку нового языка
 var layouts = appSettings.layoutsForLanguage(model[curr
 if (layouts.length > 0) {
 appSettings.layoutId = layouts[0]
 keyboardLayout.currentLayoutId = layouts[0]
 }
 }
}
```

И покажи `onLanguageChanged` чтобы убедиться что там нет лишнего:

```
sed -n '14,28p' qml/pages/SettingsPage.qml
```



Ответил: Claude

Ответил: Claude

## Дед ИИ

?

korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost\$ sed -n '14,28p' qml/pages/SettingsPage.qml

```
Connections {
 target: appSettings
 function onLessonLanguageChanged() {
 var layouts =
 appSettings.layoutsForLanguage(appSettings.lessonLanguage)
 layoutCombo.model = layouts
 layoutCombo.currentIndex = 0
 }
 function onLayoutIdChanged() {
 var layouts =
 appSettings.layoutsForLanguage(appSettings.lessonLanguage)
 var idx = layouts.indexOf(appSettings.layoutId)
 if (idx >= 0) layoutCombo.currentIndex = idx
 }
}
```

Д

### Дед ИИ

04:24

Вижу причину — `onLessonLanguageChanged` обновляет `layoutCombo.model` и сбрасывает `currentIndex = 0`, но не сохраняет новый `layoutId`` в `appSettings`. Поэтому `appSettings.layoutId` остаётся от старого языка.

Исправь `onLessonLanguageChanged`:

```
Connections {
 target: appSettings
 function onLessonLanguageChanged() {
 var layouts = appSettings.layoutsForLanguage(appSetting
 layoutCombo.model = layouts
 layoutCombo.currentIndex = 0
 // ← явно применяем первую раскладку нового языка
 if (layouts.length > 0) {
 appSettings.layoutId = layouts[0]
 keyboardLayout.currentLayoutId = layouts[0]
 }
 }
 function onLayoutIdChanged() {
 var layouts = appSettings.layoutsForLanguage(appSetting
 var idx = layouts.indexOf(appSettings.layoutId)
 if (idx >= 0) layoutCombo.currentIndex = idx
 }
}
```

И убери дублирование из `onActivated` языка — там это теперь не нужно:

# Дед ИИ

```
cmake --build build --parallel 2>&1 | tail -5
```



Ответил: Claude

Ответил: Claude

Ответил: Claude

?



11:56

чем отличаются раскладки йцукен и русская машинопись?

Д

Дед ИИ

11:56

Йцукен vs Русская машинопись

**Йцукен** — стандартная современная компьютерная раскладка, которую используют все сегодня. Названа по первым буквам верхнего ряда.

**Русская машинопись** — раскладка, разработанная для механических печатных машинок в СССР. Отличается расположением ряда символов.

## Основные отличия

|                    | Йцукен                     | Машинопись                  |
|--------------------|----------------------------|-----------------------------|
| Происхождение      | Компьютерная эра           | Печатные машинки СССР       |
| Распространённость | Повсеместно                | Редко, спецназначение       |
| Буква Ё            | Левый верхний угол (над Й) | Может быть в другом месте   |
| Знаки препинания   | Стандартные позиции        | Смещены                     |
| Эргономика         | Не оптимизирована          | Оптимизирована под механику |

## Практический вывод

💡 Для обычного пользователя разница несущественна. Машинопись актуальна только если человек работал на советских печатных машинках и переучиваться не хочет.

Если в твоём приложении нет реальной базы уроков под машинопись — **можно смело убрать** эту раскладку и не путать пользователей. Оставь только **йцукен** для RU и **qwerty** для EN.

Ответил: Claude

# Дед ИИ

?

мне нужны конкретные отличия в расположении клавиш

Д

Дед ИИ 11:58

Покажи файлы раскладок:

```
find . -name '*.json' -o -name '*.xml' -o -name '*.qml' | xargs
```



Ответил: Claude

Ответил: Claude

Ответил: Claude

Заново

Озвучить

?

да не у меня, а как оно должно быть 11:58

Д

Дед ИИ 11:58

Конкретные отличия зависят от стандарта, но классическая **русская машинопись (ГОСТ)** отличается от йцукен в нескольких местах:

**Верхний ряд цифр**

| Позиция | Йцукен | Машинопись |  
|---|---|---|  
| 1 | 1 / ! | 1 / ! |  
| 2 | 2 / " | 2 / " |  
| Ё | над Й (отдельная) | нет отдельной клавиши, Ё печаталась как Е + диакритик |

**Буквенные ряды**

| Позиция | Йцукен | Машинопись |  
|---|---|---|  
| Где x | после ш | там же |  
| Где э | после ш в среднем ряду | перемещена |  
| Где ь | после ь | убрана или перемещена — на машинке Ъ была редкой |  
| Где ж | средний ряд | там же |

⚠ Честный ответ: \*\*единого стандарта "русской машинописи" нет\*\*. Разные производители машинок (Ятрань, Москва, Украина) делали по-разному. В компьютерном мире под "машинописью" часто понимают просто \*\*JCUKEN\*\* — транслитерационную раскладку где русские буквы на тех же позициях что и фонетически близкие латинские (Й→J, Ц→C, У→U...).

## Дед ИИ

Скорее всего в твоей раскладке **typewriter** — это именно **фонетическая / JCUKEN**, а не историческая машинопись. Покажи файл раскладки:

```
find . -not -path './build/*' | xargs grep -l 'typewriter|jcuke
```



Ответил: Claude



Ответил: Claude

Заново

Озвучить

?



11:59

а какие особенности у макинтоша?

Д

Дед ИИ

12:00

Раскладка Mac (Apple)

Основные буквы **полностью совпадают** с йцукен — никаких отличий в буквенных рядах нет.

Отличия только в **символах на цифровом ряду и спецзнаках**:

| Клавиша | Windows/Linux | Mac |

| ---|---|---|

| Shift+2 | " | @ |

| Shift+3 | № | # |

| Shift+4 | ; | \$ |

| Shift+6 | : | ^ |

| Shift+7 | ? | & |

| Shift+@ (Shift+2) | " | " |

| Э шифт | — | разные кавычки |

**Практический вывод для твоего приложения**

💡 Если ты учишь **\*\*набору текста\*\***, а не слепой печати спецсимволов — раскладка Mac почти бессмысленна как отдельная сущность. Буквы те же, отличаются только символы которые в уроках вряд ли встречаются.

Скорее всего стоит **оставить только йцукен** для RU, убрать машинопись и Mac — пользователи только запутаются. Или оставить как "продвинутую" настройку скрытой по умолчанию.



Ответил: Claude

# Дед ИИ

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |       |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| ? | <br>а буква ё там есть?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 12:00 |
| Д | <b>Дед ИИ</b><br> « Claude » — лимит на сегодня исчерпан. Перевёл на «  ChatGPT».<br><br><b>Да, на Mac буква `Ё` есть</b><br><br>В русской раскладке macOS <b>Ё</b> обычно находится на той же физической клавише, что и в Windows/Linux — слева сверху, под `Esc`, там где тильда/обратная кавычка в английской раскладке.<br><br><b>Как вводится</b> <ul style="list-style-type: none"><li>– <b>ё</b> — просто нажать эту клавишу</li><li>– <b>Ё</b> — <b>Shift</b> + эта клавиша</li></ul><br><b>То есть по буквам</b><br>Для русской раскладки Apple: <ul style="list-style-type: none"><li>– буквы почти совпадают с йцукен</li><li>– <b>`Ё`</b> никуда не пропадает</li><li>– основные отличия Mac — это <b>спецсимволы</b>, а не буквы</li></ul><br> Если делаешь клавиатуру в приложении, <b>`Ё`</b> для Mac-версии русской раскладки лучше оставить на стандартном месте. | 12:00 |
|   |  Ответил: ChatGPT<br> Заново  Озвучить                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |       |
| ? | <br>ты помнишь весь контекст нашего разговора?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 12:01 |
| Д | <b>Дед ИИ</b><br><b>Да</b><br><br>Помню контекст <b>этой текущей переписки</b> : мы обсуждали твой проект <b>TypeBoost</b> , правки в <b>QML/C++</b> , календарь выбора дат, поведение переключателей в настройках, переключение раскладок при смене языка и затем различия раскладок.<br><br>Но есть важное уточнение: <ul style="list-style-type: none"><li>– я <b>помню сообщения внутри этого текущего чата</b></li><li>– я <b>не храню память между разными чатами/сессиями</b>, если платформа специально это не поддерживает</li></ul><br>То есть сейчас я могу опираться на всё, что уже было написано выше. Если хочешь, я могу прямо сейчас <b>кратко собрать весь</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 12:01 |

Дед ИИ

Заново

Озвучить

?  12:08  
 Режим

Д **Дед ИИ** 12:08  
 Режим ИИ

Сейчас:  ChatGPT

Универсальный лидер от OpenAI — сильна во всём

- ✓  ChatGPT · 98/100
- ✨ Gemini · 150/150
- 🔍 Perplexity · 80/80
- 💡 Claude · 0/100
- 💜 Базовый ∞
- ✕ Закрыть

?  12:10  
[/settings](#)

Д **Дед ИИ** 12:10  
 Настройки

-  Режим ·  ChatGPT
-  Голос · Опух — мужской низкий
-  Тариф · Макс
-  Подписка: до 31 мая 2026 · осталось 3 дня

ID: a980b5b4-acb4-561d-a96f-2aeb535774c0

-  Настроить голос
-  Подпись модели: **вкл** ✓
-  Кнопка «Заново»: **вкл** ✓
-  Кнопка «Озвучить»: **вкл** ✓
- ❓ [Помощь](#)
- ✕ Закрыть

12:10

Voice message  
Not included, change data exporting settings to download.

Дед ИИ

Voice message  
Not included, change data exporting settings to download.  
00:00, 728.2 KB

12:12

Voice message  
Not included, change data exporting settings to download.  
00:00, 318.0 KB

12:12

Voice message  
Not included, change data exporting settings to download.  
00:00, 752.6 KB

12:13

Voice message  
Not included, change data exporting settings to download.  
00:00, 751.8 KB

?   
 Режим

12:13

Д **Дед ИИ**  
 **Режим ИИ**

12:13

Сейчас:  ChatGPT

Универсальный лидер от OpenAI — сильна во всём

✓  ChatGPT · 98/100

 Gemini · 150/150

 Perplexity · 80/80

 Claude · 0/100

 Базовый ∞

✕ Закрыть

?   
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);  
  
как мне найти эту директорию?

12:17



# Дед ИИ

`QStandardPaths::writableLocation(QStandardPaths::AppDataLocation)`

возвращает **полный путь к каталогу данных приложения** для текущей ОС и текущего приложения.

**Быстро вывести путь в коде**

Добавь временно:

```
#include <QDebug>
#include <QStandardPaths>

QDebug() << QStandardPaths::writableLocation(QStandardPaths::Ap
```



Потом запусти приложение и посмотри вывод в консоли.

**Где она обычно находится**

**Linux**

Обычно так:

```
~/.local/share/<OrganizationName>/<ApplicationName>
```

или иногда:

```
~/.local/share/<ApplicationName>
```

**Windows**

Обычно так:

```
C:/Users/<Имя>/AppData/Local/<OrganizationName>/<ApplicationName>
```



или:

```
C:/Users/<Имя>/AppData/Roaming/<OrganizationName>/<ApplicationName>
```



**macOS**

Обычно так:

```
~/Library/Application Support/<ApplicationName>
```

# Дед ИИ

---

## От чего зависит точный путь

Qt формирует его с учётом:

- `QCoreApplication::setOrganizationName(...)`
- `QCoreApplication::setApplicationName(...)`
- иногда `setOrganizationDomain(...)`

Например:

```
QCoreApplication::setOrganizationName("MyCompany");
QCoreApplication::setApplicationName("TypeBoost");
```

тогда на Linux это часто будет что-то вроде:

```
/home/user/.local/share/MyCompany/TypeBoost
```

---

## Как открыть её вручную

### Linux / WSL

Если приложение запущено в Linux-среде:

```
xdg-open "$(python3 - <<'PY'
import os
print(os.path.expanduser('~/.local/share'))
PY
)"
```

Но проще сначала узнать точный путь через `qDebug()` .

Если уже знаешь путь:

```
ls -la "/путь/к/директории"
```

---

## Если хочешь без запуска приложения

Можно поискать по имени приложения:

```
find ~/.local/share -maxdepth 3 -iname '*TypeBoost*'
```

Если у тебя в коде задан `OrganizationName` , можно искать и по нему:

## Дед ИИ

### Для твоего случая

Судя по терминалу у тебя Linux/WSL, значит скорее всего путь будет примерно такой:

```
/home/korneploddviktor/.local/share/<...>/TypeBoost
```

или

```
/home/korneploddviktor/.local/share/TypeBoost
```

### Самый надёжный способ

```
qDebug() << QStandardPaths::writableLocation(QStandardPaths::Ap
```

Если хочешь, я могу сразу подсказать, в какой файл `main.cpp` это **вставить** и какой командой потом проверить вывод.

🗣️ Ответил: ChatGPT

🔄 Заново

🔊 Озвучить

? 🧑 12:21  
почему на всех гистограммах со статистикой не подписана дата 26 мая, а все остальные есть?

Д **Дед ИИ** 12:21  
Скорее всего причина одна из двух

#### 1. Подписи оси X автоматически **\*\*прорежаются\*\***

На гистограммах часто не показывают **каждую** дату, если места мало. Тогда одна дата может “выпасть”, например:

22 23 24 25 27 28

а столбец за 26 мая при этом **есть**, просто **подпись под ним скрыта**.

Это обычно происходит если:

- используется `BarCategoryAxis`
- или даты задаются списком, а Qt/chart-компонент сам решает, что не все подписи поместятся
- или ты вручную показываешь, например, каждую `n`-ю метку

# Дед ИИ

---

данных/меток.

Очень частая проблема:

- в QML дата хранится как локальная
- потом превращается в ISO/UTC
- на границе суток запись за 26 мая может стать 25 мая 21:00 или 27 мая 00:00 по UTC
- в итоге на графике группировка идёт не по той дате, которую ты ожидаешь

Тогда может быть так, что:

- визуально столбики есть
- но одна конкретная дата пропадает или сливается с соседней

---

## Что наиболее вероятно в твоём случае

Раз ты пишешь:

**\*\*на всех гистограммах\*\*** не подписана дата 26 мая, а остальные есть

это больше похоже именно на **прореживание подписей оси**, а не на потерю данных.

Если бы была ошибка в данных, обычно пропадал бы сам столбец или ломался бы только один график.

---

## Как быстро проверить

**Проверь: столбец за `26 мая` вообще есть?**

Если **столбец есть, а подписи нет** → проблема в рендеринге оси X.

Если **и столбца нет** → проблема в данных/группировке дат.

---

## Где искать в коде

Нужно посмотреть:

- как строится список дат для графика
- как задаются подписи оси X
- не скрываются ли некоторые labels специально

Особенно ищи что-то вроде:

- `axisX`
- `categories`
- `BarCategoryAxis`
- `DateTimeAxis`
- `tickCount`
- `labelsAngle`

# Дед ИИ

---

## Очень вероятный баг по твоему проекту

У тебя в `StatisticsEngine` даты для графика, вероятно, формируются как ISO-строка по дням.

Если где-то дальше в QML есть преобразование типа:

```
new Date(point.date)
```

или:

```
Qt.formatDate(new Date(point.date), "d MMM")
```

то строка вида:

```
2024-05-26
```

в JS может интерпретироваться как **UTC-дата**, и в локальном часовом поясе сместиться на предыдущий день.

Это классическая причина странностей с подписями дат.

---

## Что я бы проверил первым делом

1. Есть ли в данных запись за 26 мая
2. Есть ли столбец за 26 мая
3. Как именно подписывается ось
4. Нет ли такого:

```
new Date("2024-05-26")
```

лучше делать локально:

```
var parts = point.date.split("-")
var d = new Date(parts[0], parts[1] - 1, parts[2])
```

---

## Самый полезный следующий шаг

Покажи код, где строится гистограмма в `ProgressPage.qml` или отдельном компоненте графика.

Лучше всего так:

# Дед ИИ

или, если график точно в одном файле:

```
sed -n '1,260p' qml/pages/ProgressPage.qml
```

Тогда я скажу точно, почему именно выпадает **26 мая**.



Ответил: ChatGPT

Заново

Озвучить

?



12:26

столбик за эту дату есть. со всеми остальными датами все в порядке – есть и столбики, и подписи дат  
куда мне посмотреть в коде чтобы проверить, отображается ли дата?

Д

**Дед ИИ**

12:26

**Куда смотреть в коде**

Если **столбик за 26 мая есть**, а **подписи только у него нет**, то почти наверняка проблема **не в данных**, а в **отрисовке подписей оси X**.

Проверь 3 места:

## 1. Где формируется список дат для графика

Ищи вызов **dailyDataForRange** или **dailyDataLastDays**.

```
grep -RInE 'dailyDataForRange|dailyDataLastDays|summaryForRange
```



Там нужно понять:

- приходит ли дата **2024-05-26** в модель
- не фильтруется ли она потом в QML

## 2. Где строится сама гистограмма

Ищи компоненты графика:

```
grep -RInE 'ChartView|BarSeries|BarCategoryAxis|DateTimeAxis|Ca
```



Особенно интересны:

- **ChartView**
- **BarSeries**
- **BarCategoryAxis**
- **Repeater**, если ось рисуется вручную

# Дед ИИ

## 3. Где форматируется подпись даты

Ищи:

```
grep -RInE 'Qt.formatDate|new Date(|toLocaleDateString|split("-
```



Там может быть логика вида:

- показывать не каждую дату
- скрывать часть меток
- форматировать даты вручную

## Что именно искать

### Вариант А: подписи прореживаются специально

Например что-то вроде:

```
visible: index % 2 === 0
```

или:

```
text: index % step === 0 ? model.date : ""
```

или:

```
if (i % 3 !== 0) continue;
```

Тогда одна конкретная дата может не попасть в видимые метки.

### Вариант В: подпись рисуется, но обрезается

Ищи свойства вроде:

- `clip: true`
- маленькая `width`
- `elide: Text.ElideRight`
- поворот `rotation`
- слишком плотный `spacing`

Например:

```
Text {
 width: 20
 elide: Text.ElideRight
}
```

## Дед ИИ

---

Вариант С: дата есть в модели, но текст для неё пустой

Ищи что-то вроде:

```
text: formatLabel(model.date)
```

и потом функцию:

```
function formatLabel(dateStr) {
 ...
 return ""
}
```

---

Самая полезная проверка прямо сейчас

Сделай поиск по файлу прогресса и компонентам:

```
grep -RInE 'dailyData|ChartView|BarSeries|BarCategoryAxis|DateT
```



А потом открой найденный кусок, например:

```
sed -n '1,260p' qml/pages/ProgressPage.qml
```

если график там, или файл компонента графика, который найдётся через `grep`.

---

Если хочешь проверить именно “рисуется ли дата 26 мая”

Ищи место, где создаются подписи оси, и временно вставь вывод в лог.

Если это `Repeater` по датам, можно сделать так:

```
Component.onCompleted: console.log("label", index, modelData.da
```



или если это объект `point` :



# Дед ИИ

Если увидишь в логе **2024-05-26** , значит дата **доходит до слоя отрисовки**. Тогда проблема уже в **visible/width/elide/spacing** .

**Коротко: куда смотреть первым**

1. ``ProgressPage.qml``
2. Компонент, где рисуется **бар-чарт**
3. Всё, что связано с:
  - `BarCategoryAxis`
  - `labels`
  - `Qt.formatDate(...)`
  - `Repeater` по датам

**Дай мне это**

Лучше всего пришли вывод:

```
grep -RInE 'dailyData|ChartView|BarSeries|BarCategoryAxis|DateT
```

Тогда я скажу точно, **в какой строке проверять, почему 26 мая без подписи**.



Ответил: **ChatGPT**

12:26

**Заново**

**Озвучить**

?



12:29

korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost\$ `grep -RInE 'dailyDataForRange|dailyDataLastDays|summaryForRange|summaryLastDays' qml src`

qml/pages/ProgressPage.qml:23: property var summary:

statisticsEngine.summaryForRange(

qml/pages/ProgressPage.qml:29: summary =

statisticsEngine.summaryForRange(from, to)

qml/pages/HomePage.qml:62: var s =

statisticsEngine.summaryLastDays(1)

qml/pages/HomePage.qml:74: var s =

statisticsEngine.summaryLastDays(7)

qml/pages/HomePage.qml:86: var s =

statisticsEngine.summaryLastDays(7)

src/models/ProgressModel.cpp:29: const QVariantList raw =

m\_engine->dailyDataForRange(from, to);

src/core/StatisticsEngine.h:37: Q\_INVOKABLE QVariantMap

summaryForRange(const QDateTime &from,

src/core/StatisticsEngine.h:50: Q\_INVOKABLE QVariantList

dailyDataForRange(const QDateTime &from,

src/core/StatisticsEngine.h:54: Q\_INVOKABLE QVariantMap

summaryLastDays(int days) const;

## Дед ИИ

```
StatisticsEngine::summaryLastDays(int days) const
src/core/StatisticsEngine.cpp:50:QVariantList
StatisticsEngine::dailyDataForRange(const QDateTime &from,
src/core/StatisticsEngine.cpp:84:QVariantMap
StatisticsEngine::summaryLastDays(int days) const
src/core/StatisticsEngine.cpp:88: return summaryForRange(from, to);
src/core/StatisticsEngine.cpp:91:QVariantList
StatisticsEngine::dailyDataLastDays(int days) const
src/core/StatisticsEngine.cpp:95: return dailyDataForRange(from, to);
korneploddiviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -rInE
'ChartView|BarSeries|BarCategoryAxis|DateTimeAxis|CategoryAxis|Repe
ater|Canvas' qml
qml/components/TypingArea.qml:156: Repeater {
qml/components/NavBar.qml:37: Repeater {
qml/components/KeyboardView.qml:91: Repeater {
qml/components/KeyboardView.qml:100: Repeater {
qml/components/DatePickerButton.qml:117: Repeater {
qml/components/ProgressChart.qml:16: Canvas {
qml/pages/ProgressPage.qml:64: Repeater {
qml/pages/ProgressPage.qml:129: Repeater {
qml/pages/ProgressPage.qml:181: Repeater {
korneploddiviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -rInE
'Qt.formatDate|new Date(|toLocaleDateString|split("-")|label|labels' qml
grep: Unmatched (or \(
```

Д

Дед ИИ

12:29

Смотреть нужно в `ProgressChart.qml`

У тебя график рисуется не через Qt Charts, а вручную через `Canvas`:

```
- qml/components/ProgressChart.qml:16: Canvas
```

Значит подписи дат, скорее всего, рисуются **вручную в JS внутри `onPaint`**. Именно там и надо искать логику, почему одна подпись пропускается.

Что открыть

Сначала покажи весь компонент графика:

```
sed -n '1,240p' qml/components/ProgressChart.qml
```

Если файл длиннее:

```
sed -n '1,320p' qml/components/ProgressChart.qml
```

Почему я почти уверен, что проблема там

Раз:

## Дед ИИ

значит очень вероятно, что в `ProgressChart.qml` есть что-то вроде:

- рисовать подпись не для каждого столбца
- пропускать часть меток по шагу
- особая логика для первой/последней/чётной/нечётной даты
- проверка на пересечение текста

Типичные места:

```
if (i % 2 === 0) { ... }
```

или

```
if (x - lastLabelX < minSpacing) return;
```

или

```
if (labelX + textWidth > width) ...
```

### Дополнительно полезно проверить данные

Раз есть `ProgressModel.cpp`, можно ещё посмотреть, не преобразуется ли дата там необычно:

```
sed -n '1,220p' src/models/ProgressModel.cpp
```

Но первым делом именно `ProgressChart.qml`.

### Ещё одна мелочь

Команда с `grep` упала из-за скобки. Правильный вариант:

```
grep -RInE 'Qt.formatDate|new Date|toLocaleDateString|split("-"
```

Но сейчас это уже вторично — главный кандидат найден.

Пришли содержимое `qml/components/ProgressChart.qml`, и я скажу точно, где исчезает подпись `26 мая`.



Ответил: ChatGPT

Заново

Озвучить



# Дед ИИ

Д

Дед ИИ

12:30

Причина найдена

В `ProgressChart.qml` подписи дат рисуются не для всех столбцов, а только для трёх:

- первого
- среднего
- последнего

Вот этот кусок:

```
if (k === 0 || k === Math.floor(rowCount / 2) || k === rowCount
 ctx.fillStyle = Theme.textSecondary;
 ctx.textAlign = "center";
 ctx.fillText(p.date, barX + (barWidth / 2), height - paddin
}
```

То есть дата `26 мая` будет подписана **только если** она попала в один из этих индексов.  
Если нет — столбик есть, а подписи нет.

Почему тебе казалось, что остальные даты подписаны

Скорее всего:

- на коротких диапазонах `26 мая` иногда не попадает в `0`, `middle`, `last`
- а соседние даты попадают
- из-за этого выглядит как будто “все есть, кроме 26 мая”

Но по коду сейчас логика именно такая: **не все даты подписываются вообще.**

Что можно сделать

Вариант 1 — рисовать подпись для каждого столбца

Если столбцов немного:

```
ctx.fillStyle = Theme.textSecondary;
ctx.textAlign = "center";
ctx.fillText(p.date, barX + (barWidth / 2), height - paddingBot
```

То есть просто убрать `if`.

## Дед ИИ

Вариант 2 — рисовать каждую палочку с датой

Если дат много, это лучше. Например, считать шаг автоматически:

```
var labelStep = Math.max(1, Math.ceil(rowCount / 6));
```

И потом:

```
if (k % labelStep === 0 || k === rowCount - 1) {
 ctx.fillStyle = Theme.textSecondary;
 ctx.textAlign = "center";
 ctx.fillText(p.date, barX + (barWidth / 2), height - paddin
}
```

Так будет примерно 5-7 подписей, а не только 3.

Что я бы рекомендовал

Сделать **авто-шаг**. Тогда подписи будут стабильнее и понятнее.

Пример замены блока в `onPaint` :

```
var labelStep = Math.max(1, Math.ceil(rowCount / 6));

for (var k = 0; k < rowCount; k++) {
 var p = points[k];
 var barHeight = (p.value / maxVal) * graphHeight;
 var barX = paddingLeft + (k * (barWidth + barSpacing))
 var barY = height - paddingBottom - barHeight;

 if (root.activeMetric === "errorRate") {
 ctx.fillStyle = p.value > 5 ? Theme.errorColor : Theme.
 } else {
 ctx.fillStyle = Theme.primary;
 }

 ctx.fillRect(barX, barY, barWidth, barHeight);

 if (k % labelStep === 0 || k === rowCount - 1) {
 ctx.fillStyle = Theme.textSecondary;
 ctx.textAlign = "center";
 ctx.fillText(p.date, barX + (barWidth / 2), height - pa
 }
}
```

## Дед ИИ

подписей могут палезать. Лучше убедитесь, что дата уже короткая, например:

- 26.05
- 26 мая

Если хочешь, я могу сразу предложить **готовую улучшенную версию** `ProgressChart.qml`:

- с нормальным шагом подписей,
- поворотом текста,
- и более аккуратной отрисовкой оси X.

 [Заново](#)

 [Озвучить](#)

?



12:33

замени смайлик  каким-нибудь таким, который будет отображаться

Д

**Дед ИИ**

12:33

**Подойдут такие варианты**

Если  не отображается, чаще всего надёжнее заменить его на более простой Unicode-символ:

- `▲` — самый безопасный вариант
- `■` — похоже на столбик диаграммы
- `▲` — визуально напоминает рост
- `↑` — если нужен смысл “рост/вверх”

**Я бы рекомендовал**

`▲` — обычно отображается почти везде.

---

### Пример замены

Было:

text: "

Сделай так:

text: "▲"

или, если нужен “бар-чарт” стиль:

text: "■"

# Дед ИИ

символы:

- статистика: `▯`, `▮`, `▴`
- прогресс: `↗`
- график: `≈` или `▣` как условная иконка

Что я бы поставил в твоём случае

Для страницы прогресса/статистики:

text: "↗"

или

text: "▴"

Если хочешь, я могу сразу подобрать **замены для всех емоji в интерфейсе**, чтобы весь UI выглядел единообразно.

 Ответил: ChatGPT

 Заново

 Озвучить

?  13:27

 Режим

Д **Дед ИИ** 13:27

 Режим ИИ

Сейчас:  ChatGPT

Универсальный лидер от OpenAI — сильна во всём

  ChatGPT · 92/100

 Gemini · 150/150

 Perplexity · 80/80

 Claude · 0/100

 Базовый ∞

 Закрыть

?  14:21

 Режим

Д **Дед ИИ** 14:21

 Режим ИИ

Дед ИИ

- ✓ 🤖 ChatGPT · 92/100
- ✨ Gemini · 150/150
- 🔍 Perplexity · 80/80
- 💡 Claude · 0/100
- 💬 Базовый ∞
- ✕ Заккрыть

?

  
[/help](#)

14:21

Д

Дед ИИ

🔍 Дед ИИ — справка

💬 Текст — просто напишите вопрос

🎤 Голос — отправьте голосовое

🎨 Картинки — напишите «нарисуй...»

📁 Файлы — отправьте фото или PDF

Команды:

[/start](#) — главное меню

[/new](#) — новый диалог

[/context](#) — что я сейчас помню

[/mode](#) — сменить режим AI

[/balance](#) — что осталось

[/sub](#) — подписка

[/voice](#) — голос Деда

[/settings](#) — настройки

[/support](#) — связаться с поддержкой

🌐 Веб-поиск: включите кнопкой 🌐 в [/mode](#) — модель найдёт актуальную информацию и приведёт ссылки на источники.

Память бота: помню весь разговор, пока вы не начнёте новый чат. Чтобы переписать конкретный ответ — нажмите на него и выберите «🔄 Заново»: я отвечу с нуля, а всё что шло после — удалится.

💬 Связаться с поддержкой

✕ Заккрыть

14:21

?

  
[/balance](#)

14:22

Д

Дед ИИ

 Моя подписка

14:22

Макс · 1 990 Р / мес

⚠️ Автопродление отменено



Дед ИИ

|   |                                                                                                                                                                                                                                                                                                                                 |       |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| ? | <div><div></div><div>Режим</div></div>                                                                                                                                                                                                                                                                                          | 17:19 |
| Д | <div><div>Дед ИИ</div><div><div>Режим ИИ</div><div>Сейчас:  ChatGPT</div><div>Универсальный лидер от OpenAI — сильна во всём</div><div><div><div>✓  ChatGPT · 92/100</div><div> Gemini · 150/150</div><div> Perplexity · 80/80</div><div> Claude · 0/100</div><div> Базовый ∞</div><div>× Закрыть</div></div></div></div></div> | 17:19 |
| ? | <div><div></div><div>/mode</div></div>                                                                                                                                                                                                                                                                                          | 18:42 |
| ? | <div><div></div><div>Режим</div></div>                                                                                                                                                                                                                                                                                          | 20:53 |
| Д | <div><div>Дед ИИ</div><div><div>Режим ИИ</div><div>Сейчас:  ChatGPT</div><div>Универсальный лидер от OpenAI — сильна во всём</div><div><div><div>✓  ChatGPT · 92/100</div><div> Gemini · 150/150</div><div> Perplexity · 80/80</div><div> Claude · 0/100</div><div> Базовый ∞</div><div>× Закрыть</div></div></div></div></div> | 20:53 |
| ? | <div><div></div><div>/sub</div></div>                                                                                                                                                                                                                                                                                           | 20:55 |
| Д | <div><div>Дед ИИ</div><div><div>Моя подписка</div></div></div>                                                                                                                                                                                                                                                                  | 20:55 |

# Дед ИИ

После — тариф станет бесплатным.

✕ Закрыть

?



/help

20:55

Д

Дед ИИ

? Дед ИИ — справка

💬 Текст — просто напишите вопрос

🎤 Голос — отправьте голосовое

🖼️ Картинки — напишите «нарисуй...»

📎 Файлы — отправьте фото или PDF

Команды:

/start — главное меню

/new — новый диалог

/context — что я сейчас помню

/mode — сменить режим AI

/balance — что осталось

/sub — подписка

/voice — голос Деда

/settings — настройки

/support — связаться с поддержкой

🌐 Веб-поиск: включите кнопкой 🌐 в /mode — модель найдёт актуальную информацию и приведёт ссылки на источники.

Память бота: помню весь разговор, пока вы не начнёте новый чат. Чтобы переписать конкретный ответ — нажмите на него и выберите «🔄 Заново»: я отвечу с нуля, а всё что шло после — удалится.

💬 Связаться с поддержкой

20:55

✕ Закрыть

?



/balance

20:55

Д

Дед ИИ

📊 Осталось на сегодня:

Базовые

💬 Базовый — ∞

Топовые ИИ

🤖 ChatGPT — 92 из 100

🌟 Gemini — 150 из 150

🔍 Perplexity — 80 из 80

💡 Claude — 0 из 100

🖼️ Картинки — 150 из 150

20:55

file:///C:/Users/demak/Downloads/Telegram Desktop/ChatExport\_2026-05-29/messages2.html

34/82

Дед ИИ

Сброс через 1 ч 4 мин

- 📄 Моя подписка
- 🤖 Сменить режим
- ✕ Закрыть

?

👤

/balance

22:35

Д

Дед ИИ

📊 Осталось на сегодня:

Базовые

💬 Базовый — ∞

Топовые ИИ

🤖 ChatGPT — 92 из 100

🌟 Gemini — 150 из 150

🔍 Perplexity — 80 из 80

💬 Claude — 0 из 100

🎨 Картинки — 150 из 150

Тариф: Макс

Подписка: до 31 мая 2026 · осталось 3 дня

Сброс: через 2 ч 24 мин

📄 Моя подписка

🤖 Сменить режим

✕ Закрыть

29 May 2026

?

👤

/balance

07:58

Д

Дед ИИ

📊 Осталось на сегодня:

Базовые

💬 Базовый — ∞

Топовые ИИ

🤖 ChatGPT — 100 из 100

🌟 Gemini — 150 из 150

🔍 Perplexity — 80 из 80

💬 Claude — 100 из 100

🎨 Картинки — 150 из 150

Тариф: Макс

file:///C:/Users/demak/Downloads/Telegram Desktop/ChatExport\_2026-05-29/messages2.html

35/82

Дед ИИ

🤖 Сменить режим  
✕ Закрыть

? 🧑 07:58  
🤖 Режим  
/context 07:59

Д Дед ИИ 07:59  
📌 Текущий разговор  
  
Сообщений в памяти: 30 (твоих реплик: 15)  
  
Контекст не сбрасывается по времени.  
/new — начать новый разговор.  
  
🔄 Переключил на 🧑 Claude без очистки. 07:59  
Эта модель попыбует ответить с учётом старого разговора.

? 🧑 08:00  
сформулируй контекст нашей переписки

Д Дед ИИ 08:00  
Контекст переписки  
  
Проект  
TypeBoost — приложение для обучения слепой печати на Qt/QML + C++.

Что обсуждали технически

- 1. Переключение раскладок при смене языка (`SettingsPage.qml`)
  - При смене языка в `languageCombo` вызывался `onLessonLanguageChanged`
  - Он обновлял `layoutCombo.model` и сбрасывал `currentIndex = 0`, но не сохранял новый `layoutId` в `appSettings`
  - Решение: добавить явное обновление `appSettings.layoutId` и `keyboardLayout.currentLayoutId` в `onLessonLanguageChanged`
- 2. Пропавшая подпись даты `26 мая` на гистограммах (`ProgressChart.qml`)
  - График рисуется вручную через `Canvas`
  - Подписи дат рисовались только для 3 столбцов: первого, среднего, последнего
  - Решение: заменить логику на авто-шаг `Math.ceil(rowCount / 6)` чтобы показывать ~5-7 подписей равномерно
- 3. `QStandardPaths::AppDataLocation``
  - Как найти директорию данных приложения
  - На Linux это обычно `~/.local/share/<OrgName>/<AppName>`

# Дед ИИ

## Что обсуждали теоретически

- Отличия раскладок **йцукен** / **русская машинопись** / **Mac**
- Буква **Ё** в раскладке Mac — есть, на том же месте что и в Windows

## Архитектура проекта (что видно из кода)

- QML: `SettingsPage.qml` , `ProgressPage.qml` , `ProgressChart.qml` , `HomePage.qml`
- C++: `StatisticsEngine` , `ProgressModel`
- Методы: `dailyDataForRange` , `summaryForRange` , `summaryLastDays` , `dailyDataLastDays`
- Настройки: `appSettings` — `layoutId` , `lessonLanguage` , `layoutsForLanguage()`
- График: кастомный `Canvas` в `ProgressChart.qml` , метрика `activeMetric` (в т.ч. `errorRate` )



Ответил: Claude

Заново

Озвучить

?



08:07

смотри, приложение уже почти рабочее. осталось исправить последнюю грубую ошибку, и будет хорошо

ошибка такая. если взять любой текст, написать его полностью неправильно (например просто нажать одну букву и удерживать), то чем больше будет неправильных символов, тем меньше будет процент ошибок. это явная логическая ошибка. например, если надо напечатать текст «Привет, мир!», а пользователь ввел 100 букв «а», то текущий процент ошибок будет где-то в районе 5%. и чем больше ввести этих символов, тем меньше будет процент. короче говоря, он будет стремиться к нулю. но это еще не вся беда. новая проблема возникает тут же, если этот же огромный неправильный текст стереть. процент ошибок устремится в бесконечность (будет около 5000).

сейчас твоя задача – разобраться как программа вычисляет этот процент. а затем исправить подсчет.

для начала изложи, как ты понимаешь сам концепцию подсчета этого процента. сначала я хочу утвердить эту формулу, а потом уже надо будет приступать к исправлениям в коде

Д

**Дед ИИ**

08:08

Как должен считаться процент ошибок

## Суть проблемы

Твое описание чётко указывает на то, что сейчас программа считает примерно так:

# Дед ИИ

где `totalTyped` — общее количество **введённых** символов (включая лишние). Когда пользователь вводит 100 лишних букв — знаменатель растёт, процент падает. Когда стирает — знаменатель уменьшается, процент улетает вверх.

---

## Как должно быть правильно

Процент ошибок — это **доля позиций в эталонном тексте, где пользователь ошибся**.

Формула:

```
errorRate = wrongPositions / len(referenceText) * 100
```

Где:

- `referenceText` — эталонный текст урока (фиксированная длина, **не меняется**)
- `wrongPositions` — количество позиций, где введённый символ **не совпал** с эталонным

---

## Что считается ошибкой

| Ситуация | Ошибка? |

|---|---|

| Пользователь ввёл правильный символ |  нет |

| Пользователь ввёл неправильный символ |  да |

| Позиция ещё не достигнута (не введена) |  нет, просто не считается |

| Пользователь ввёл **лишние** символы сверх длины эталона | игнорируются |

---

## Ключевое свойство правильной формулы

 Знаменатель — это **\*\*длина эталонного текста\*\***, и он **\*\*никогда не меняется\*\***. Пользователь не может повлиять на знаменатель, сколько бы символов он ни вводил.

Тогда:

- вводишь 100 лишних букв → они игнорируются → процент не меняется
- стираешь → уменьшается число проверенных позиций → процент пересчитывается честно

## Дед ИИ

Предлагаю считать так:

```
checked = min(len(userInput), len(referenceText))
wrongPositions = количество позиций i в [0, checked), где userI
errorRate = wrongPositions / len(referenceText) * 100
```

Это даст:

- **0%** если всё правильно
- **100%** если каждый символ эталона введён неверно
- **не зависит** от лишних символов сверх длины эталона

Согласен с такой формулой? Тогда идём смотреть код.



Ответил: **Claude**

**Заново**

**Озвучить**

?



08:17

да, формула верная. тогда давай еще добавим валидацию на пользовательский ввод, чтобы она ограничивала его длину до длины эталонного текста. потому что с одной стороны если пользователь ввел сильно больше символов, чем надо, но в эталонном тексте ошибся, например, только в последнем символе, процент ошибок действительно будет в районе 99, но с другой стороны это может показаться нелогичным. поэтому компромисс здесь – ограничить длину ввода. тогда все будет выглядеть адекватно. что скажешь? я в чем-то заблуждаюсь?

Д

**Дед ИИ**

08:17

**Ты рассуждаешь правильно**

Ограничение длины ввода — это **не костыль**, а правильное UX-решение для тренажёра печати.

**Почему это логично**

В тренажёре печати задача пользователя — **воспроизвести эталонный текст**. Символы сверх длины эталона не имеют смысла вообще. Это не текстовый редактор, а упражнение с чётко заданным заданием.

Значит:

- ввод ограничен длиной эталона — **естественное поведение**
- лишние символы просто не принимаются — пользователь не запутается
- формула `wrongPositions / len(referenceText)` работает идеально

# Дед ИИ

## Два изменения вместе дают чистое решение

1. Ограничить длину userInput до len(referenceText)
2.  $errorRate = wrongPositions / len(referenceText) * 100$

Тогда:

- знаменатель всегда фиксирован ✓
- числитель никогда не превысит знаменатель ✓
- процент всегда в диапазоне `[0, 100]` ✓
- никаких краевых случаев ✓

## Ты ни в чём не заблуждаешься

Единственный теоретический контраргумент — "а что если пользователь хочет ввести лишний символ случайно?" Но в тренажёре это не имеет смысла: лишний символ просто игнорируется или блокируется, и пользователь видит что дошёл до конца текста.

Теперь показывай код, где считается `errorRate` и где обрабатывается ввод:

```
grep -RInE 'errorRate|wrongPositions|inputText|userInput|current
```

🖨️ Ответил: Claude

🔄 Заново

🔊 Озвучить

?



08:25

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -RInE
'errorRate|wrongPositions|inputText|userInput|currentInput|keyPress|te
xtInput' src qml --include="*.cpp" --include="*.h" --include="*.qml" |
grep -v build
src/models/SessionResult.h:34: qreal errorRate() const
src/core/TypingSession.cpp:18: QString TypingSession::inputText()
const { return m_inputText; }
src/core/TypingSession.cpp:23: qreal TypingSession::errorRate() const {
return m_errorRate; }
src/core/TypingSession.cpp:29: const int pos = m_inputText.length();
src/core/TypingSession.cpp:37: const int pos = m_inputText.length();
src/core/TypingSession.cpp:55: return m_inputText.length();
src/core/TypingSession.cpp:60: const int pos = m_inputText.length();
src/core/TypingSession.cpp:78: m_inputText = "";
src/core/TypingSession.cpp:89: m_errorRate = 0.0;
```



## Дед ИИ

```

src/core/TypingSession.cpp:145: if (text.length() >
m_inputText.length()) {
src/core/TypingSession.cpp:154: m_inputText = text;
src/core/TypingSession.cpp:155: emit inputTextChanged();
src/core/TypingSession.cpp:174: const int len =
qMin(m_inputText.length(), m_targetText.length());
src/core/TypingSession.cpp:176: if (m_inputText.at(i) ==
m_targetText.at(i))
src/core/TypingSession.cpp:182: const int total =
m_inputText.length();
src/core/TypingSession.cpp:183: m_errorRate = (total > 0) ?
(static_cast<qreal>(m_errorCount) / total * 100.0) : 0.0;
src/core/TypingSession.cpp:190: if (m_inputText.length() <
m_targetText.length()) return;
src/core/TypingSession.cpp:191: if (m_inputText != m_targetText)
return;
src/core/StatisticsEngine.cpp:25: sumErr += s.errorRate();
src/core/StatisticsEngine.cpp:72: sumErr += s.errorRate();
src/core/TypingSession.h:24: Q_PROPERTY(QString inputText READ
inputText WRITE setInputText NOTIFY inputTextChanged)
src/core/TypingSession.h:29: Q_PROPERTY(qreal errorRate READ
errorRate NOTIFY statsUpdated)
src/core/TypingSession.h:43: QString inputText() const;
src/core/TypingSession.h:49: qreal errorRate() const;
src/core/TypingSession.h:72: void inputTextChanged();
src/core/TypingSession.h:88: QString m_inputText;
src/core/TypingSession.h:104: qreal m_errorRate = 0.0;
qml/components/TypingArea.qml:31: root.cachedInputText =
typingSession.inputText
qml/components/TypingArea.qml:35: "inputText=",
typingSession.inputText.substring(0,20),
qml/components/TypingArea.qml:78: root.cachedInputText =
typingSession.inputText
qml/components/StatisticsView.qml:26: root.cachedErrors =
typingSession.errorRate
qml/components/StatisticsView.qml:67: root.cachedErrors =
typingSession.errorRate
qml/components/StatisticsView.qml:72: if (!root.inputStarted &&
typingSession.inputText.length > 0) {
qml/components/StatisticsView.qml:75: if (typingSession.inputText
=== typingSession.targetText) {
qml/components/ProgressChart.qml:37: if (root.activeMetric ===
"errorRate") roleOffset = 5;
qml/components/ProgressChart.qml:86: if (root.activeMetric ===
"errorRate") {
qml/pages/ProgressPage.qml:185: { label: qsTr("Ошибки"), metric:
"errorRate" },
qml/pages/PracticePage.qml:43: savedInputText =
typingSession.inputText
qml/pages/LessonPage.qml:41: savedInputText =
typingSession.inputText
qml/pages/LessonPage.qml:85: root.finalErrorRate =
typingSession.errorRate;

```

## Дед ИИ

```
const int total = m_inputText.length();
m_errorRate = (total > 0) ? (static_cast<qreal>(m_errorCount) /
```

Знаменатель — `m_inputText.length()` , а должен быть `m_targetText.length()` .

Покажи весь файл:

```
sed -n '1,200p' src/core/TypingSession.cpp
```

Чтобы увидеть сразу и логику ввода (строки 138-155) и `setInputText` , и там же добавим ограничение длины.



Ответил: Claude

Заново

Озвучить

?



08:26

message.txt

6.3 KB

Д

Дед ИИ

08:26

Виджу всё что нужно. Нужно сделать **два изменения** в `TypingSession.cpp` :

**Изменение 1 — исправить формулу `errorRate`**

**Строка 183, заменить знаменатель:**

```
// БЫЛО
const int total = m_inputText.length();
m_errorRate = (total > 0) ? (static_cast<qreal>(m_errorCount) /

// СТАЛО
const int total = m_targetText.length();
m_errorRate = (total > 0) ? (static_cast<qreal>(m_errorCount) /
```

**Изменение 2 — ограничить длину ввода в `setInputText`**

## Дед ИИ

```

 if (!m_active || m_paused) return;

 // — Ограничиваем длину ввода длиной эталона —————
 const QString limited = text.left(m_targetText.length());
 if (m_inputText == limited) return; // ← сравниваем с обре

 if (!m_timerStarted && !limited.isEmpty()) {
 m_timerStarted = true;
 m_timer.restart();
 qDebug() << "[TypingSession] timer started ownerTag=" <
 }
 if (limited.length() > m_inputText.length()) {
 const int newPos = limited.length() - 1;
 if (newPos < m_targetText.length()) {
 if (limited.at(newPos) != m_targetText.at(newPos))
 ++m_errorCount;
 }
 }
}

m_inputText = limited; // ← сохраняем обрезанную версию
emit inputTextChanged();
emit cursorPosChanged();

updateStats();
checkCompletion();
}

```

### Итог двух изменений

| Ситуация | До | После |  
 |---|---|---|  
 | Ввёл 100 лишних букв |  $errorRate \rightarrow 0\%$  | ввод обрезан, лишние не  
 принимаются |  
 | Стёр лишние буквы |  $errorRate \rightarrow 5000\%$  | невозможно, лишних нет |  
 | Знаменатель  $errorRate$  | `inputText.length()` | `targetText.length()`  
 (фиксирован) |  
 | Диапазон  $errorRate$  |  $[0, \infty)$  |  $[0, 100]$   |

Применяй и собирай:

```
cmake --build build --parallel 2>&1 | tail -5
```

 Заново

 Озвучить

?



08:33

## Дед ИИ

могу вводить лишние символы в строку ввода.

2. если я ввел один символ неправильно, у меня корректно вырос процент ошибок. но если я этот символ стер, то процент ошибок не возвращается к прежнему, хотя должен

Д

Дед ИИ

08:34

Покажи полный `setInputText` и `updateStats` :

```
sed -n '135,200p' src/core/TypingSession.cpp
```

Нужно видеть точно как сейчас считается `m_errorCount` при удалении символа — скорее всего он только растёт, но никогда не уменьшается.



Ответил: Claude

Заново

Озвучить

?



08:34

```
korneploddiviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ sed -n '135,200p' src/core/TypingSession.cpp
```

```
void TypingSession::setInputText(const QString &text)
{
 if (!m_active || m_paused) return;
```

// — Ограничиваем длину ввода длиной эталона

```
const QString limited = text.left(m_targetText.length());
if (m_inputText == limited) return; // ← сравниваем с обрезанной версией
```

```
if (!m_timerStarted && !limited.isEmpty()) {
 m_timerStarted = true;
 m_timer.restart();
 qDebug() << "[TypingSession] timer started ownerTag=" << m_ownerTag;
}
if (limited.length() > m_inputText.length()) {
 const int newPos = limited.length() - 1;
 if (newPos < m_targetText.length()) {
 if (limited.at(newPos) != m_targetText.at(newPos)) {
 ++m_errorCount;
 }
 }
}
```

```
m_inputText = limited; // ← сохраняем обрезанную версию
emit inputTextChanged();
emit cursorPosChanged();
```

```
updateStats();
checkCompletion();
}
```

## Дед ИИ

---

```
void TypingSession::updateStats()
{
 if (!m_timerStarted) return;
 if (!m_timer.isValid()) return;

 const qreal elapsed = (m_pausedElapsed + m_timer.elapsed()) /
 1000.0; // seconds
 if (elapsed <= 0.0) return;

 // Count correct chars typed so far
 int correct = 0;
 const int len = qMin(m_inputText.length(), m_targetText.length());
 for (int i = 0; i < len; ++i) {
 if (m_inputText.at(i) == m_targetText.at(i))
 ++correct;
 }

 m_cpm = (correct / elapsed) * 60.0;

 const int total = m_targetText.length();
 m_errorRate = (total > 0) ? (static_cast<qreal>(m_errorCount) / total *
 100.0) : 0.0;

 emit statsUpdated();
}

void TypingSession::checkCompletion()
{
 if (m_inputText.length() < m_targetText.length()) return;
 if (m_inputText != m_targetText) return;

 // Session complete
 m_active = false;
 const qreal elapsed = m_timerStarted
 ? ((m_pausedElapsed + m_timer.elapsed()) / 1000.0)
 : 0.0;
```

Д

**Дед ИИ**

08:35

Виджу оба бага сразу.

---

### Проблема 1 — лишние символы всё ещё вводятся

`text.left(m_targetText.length())` обрезает строку **внутри C++**, но QML-компонент (`TextInput` или `TextField`) физически всё равно принимает символы. Нужно блокировать на уровне QML.

Покажи где в QML привязан ввод:

# Дед ИИ

## Проблема 2 — `m\_errorCount` только растёт

Это классическая ошибка: `m_errorCount` инкрементируется при вводе неверного символа, но **никогда не уменьшается** при удалении. Нужно полностью отказаться от счётчика `m_errorCount` и считать ошибки **прямо по текущему состоянию** в `updateStats`.

Замени в `updateStats`:

```
// БЫЛО
const int total = m_targetText.length();
m_errorRate = (total > 0) ? (static_cast<qreal>(m_errorCount) /

// СТАЛО — считаем ошибки по текущему состоянию, без m_errorCou
int errors = 0;
const int checked = m_inputText.length(); // только введенные п
for (int i = 0; i < checked; ++i) {
 if (m_inputText.at(i) != m_targetText.at(i))
 ++errors;
}
const int total = m_targetText.length();
m_errorRate = (total > 0) ? (static_cast<qreal>(errors) / total
```

⚠ После этого `m_errorCount` вообще не нужен для `errorRate`. Но проверь — возможно он используется ещё где-то (например в `SessionResult`). Проверь:

```
grep -rn 'm_errorCount' src/
```

🤖 Ответил: Claude

🔄 Заново

🗣 Озвучить

?



08:40

```
korneploddeviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -n
'setInputText|inputText|TextInput|TextField|Keys|onTextChanged|text:'
qml/components/TypingArea.qml | head -40
korneploddeviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -rn
'm_errorCount' src/
src/core/TypingSession.cpp:24:int TypingSession::errorCount() const {
return m_errorCount; }
src/core/TypingSession.cpp:79: m_errorCount = 0;
src/core/TypingSession.cpp:152: ++m_errorCount;
src/core/TypingSession.cpp:214: result.correctChars =
```

## Дед ИИ

процент ошибок теперь считается верно, осталось включить валидацию

Д

Дед ИИ

08:40

Хорошо, `m_errorCount` используется в `SessionResult` — его трогать не будем, пусть остаётся как есть для финального результата.

Теперь разберёмся с валидацией. Покажи как устроен ввод в QML:

```
cat qml/components/TypingArea.qml
```



Ответил: Claude



Ответил: Claude

Заново

Озвучить

?



08:41

**message.txt**

9.1 KB

Д

Дед ИИ

08:41

Вижу всё. Ввод идёт через `TextField`, и ограничение нужно добавить прямо в `onTextChanged` в QML — просто обрезать текст до длины эталона.

**Исправление в `TypingArea.qml`**

Найди `onTextChanged` в `TextField` и добавь одну проверку в самом начале:

```
onTextChanged: {
 console.log("[TypingArea] text=", text,
 "ownerTag=", root.ownerTag,
 "sessionOwner=", typingSession.ownerTag,
 "active=", typingSession.active,
 "paused=", typingSession.paused,
 "focus=", inputField.activeFocus)

 if (!inputField.activeFocus)
 retur
 if (!typingSession.active)
 retur
 if (typingSession.paused)
 retur
 if (typingSession.ownerTag !== root.ownerTag)
 retur
 if (typingSession.targetText === "")
 retur

 // — Ограничение длины ввода —
 const maxLen = typingSession.targetText.length
```

## Дед ИИ

```
// _____

typingSession.setInputText(text)
}
```

💡 `return` после присвоения `inputField.text` важен — QML автоматически вызовет `onTextChanged` повторно с уже обрезанным текстом, и тогда он пройдет все проверки нормально.

### Что можно убрать из C++

После этого строка в `setInputText` :

```
const QString limited = text.left(m_targetText.length());
```

становится избыточной, но **не вредит** — можно оставить как второй уровень защиты.

Собирай и проверяй:

```
cmake --build build --parallel 2>&1 | tail -5
```

 [Заново](#)

 [Озвучить](#)

?



08:52

хорошо, ввод ограничивается как надо  
но есть момент который меня не устраивает.  
если я ввел строку такой же длины, что и эталонная, но где-то совершил ошибку (неважно где и сколько), а потом редактирую свой ввод где-то посередине строки, или в ее начале (то есть короче редактирую не последний символ), но после каждого изменения курсор ввода автоматически перемещается в конец строки ввода.  
объясни, зачем так сделано

Д

**Дед ИИ**

08:52

**Почему курсор прыгает в конец**

Это происходит из-за строки в `TypingArea.qml` :

```
const maxLen = typingSession.targetText.length
if (text.length > maxLen) {
```



## Дед ИИ

Когда ты редактируешь середину строки — `text.length` уже равен `maxLen`, ограничение не срабатывает. Но курсор всё равно прыгает в конец.

Скорее всего причина в другом месте — либо в `onTextChanged`, либо где-то явно вызывается `inputField.cursorPosition = inputField.length` или `inputField.selectAll()` или `inputField.end()`.

Покажи строки вокруг `cursorPosition` и `forceActiveFocus`:

```
grep -n 'cursorPosition|forceActiveFocus|selectAll|.end()|.length' qml/components/TypingArea.qml
```

Ответил: Claude

Заново

Озвучить

?



08:54

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -En 'cursorPosition|forceActiveFocus|selectAll|.end()|.length\b' qml/components/TypingArea.qml
42: inputField.forceActiveFocus()
52: inputField.forceActiveFocus()
67: inputField.forceActiveFocus() // ← возвращаем фокус
134: const maxLen = typingSession.targetText.length
135: if (text.length > maxLen) {
144: Component.onCompleted: inputField.forceActiveFocus()
165: model: root.chars.length
169: property string typed: index < root.cachedInputText.length
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$
```

Д

Дед ИИ

08:54

Явного `cursorPosition` нет. Курсор прыгает в конец из-за `forceActiveFocus()`.

Когда `TextField` теряет и снова получает фокус — курсор автоматически встаёт в конец. А `forceActiveFocus()` вызывается в нескольких местах, в том числе, вероятно, реагируя на изменение текста.

Покажи контекст вокруг этих строк:

```
sed -n '35,75p' qml/components/TypingArea.qml
```

Ответил: Claude

Ответил: Claude

Заново

Озвучить

## Дед ИИ

```
qml/components/TypingArea.qml
"inputText=", typingSession.inputText.substring(0,20),
"cursorPos=", typingSession.cursorPos)
}

function forceInputFocus() {
syncCache()
inputField.enabled = true
inputField.forceActiveFocus()
}

function clear() {
finishOverlay.visible = false
root.cachedInputText = ""
root.cachedCursorPos = 0
inputField.enabled = true
inputField.text = "" // ← сначала текст
Qt.callLater(function() { // ← фокус чуть позже
inputField.forceActiveFocus()
})
}

// — Реакция на смену текста

onCurrentTextChanged: {
console.log("[TypingArea] currentTextChanged ownerTag=",
root.ownerTag,
"text=", currentText.substring(0, 30))
if (currentText !== "") {
root.cachedInputText = ""
root.cachedCursorPos = 0
inputField.text = "" // ← СНАЧАЛА чистим поле
typingSession.start(currentText, lessonId, language, layoutId,
ownerTag)
inputField.forceActiveFocus() // ← возвращаем фокус
}
}

// — Сигналы сессии

Connections {
target: typingSession
```

Д

Дед ИИ

08:54

Здесь `forceActiveFocus()` вызывается **только при смене текста** (`onCurrentTextChanged`) и при явном вызове `forceInputFocus()` снаружи. Это нормально — они не должны триггериться при вводе символов.

# Дед ИИ

 Ответил: Claude

 Ответил: Claude

 Заново

 Озвучить

?



08:56

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ sed -n '100,165p'
qml/components/TypingArea.qml
}
```

// — Поле ввода

```
TextField {
 id: inputField
 Layout.fillWidth: true
 font.family: Theme.monoFontFamily
 font.pixelSize: 20
 color: Theme.textPrimary
 placeholderText: qsTr("Начните печатать здесь для старта
тренировки...")

 background: Rectangle {
 color: Theme.surface
 border.color: inputField.activeFocus ? Theme.primary : Theme.divider
 border.width: inputField.activeFocus ? 2 : 1
 radius: Theme.radiusM
 }
}
```

```
onTextChanged: {
 console.log("[TypingArea] text=", text,
"ownerTag=", root.ownerTag,
"sessionOwner=", typingSession.ownerTag,
"active=", typingSession.active,
"paused=", typingSession.paused,
"focus=", inputField.activeFocus)
```

```
if (!inputField.activeFocus) return
if (!typingSession.active) return
if (typingSession.paused) return
if (typingSession.ownerTag !== root.ownerTag) return
if (typingSession.targetText === "") return
```

// — Ограничение длины ввода

```
const maxLen = typingSession.targetText.length
if (text.length > maxLen) {
 inputField.text = text.substring(0, maxLen)
 return // onTextChanged сработает повторно уже с обрезанным
```

## Дед ИИ

```
typingSession.setInputText(text)
}

Component.onCompleted: inputField.forceActiveFocus()
}

// — Отображение текста

Rectangle {
 Layout.fillWidth: true
 Layout.preferredHeight: Math.max(120, flowLayout.implicitHeight +
 Theme.spacingL * 2)
 color: Theme.surface
 border.color: Theme.divider
 border.width: 1
 radius: Theme.radiusM
 clip: true

 Flow {
 id: flowLayout
 anchors.fill: parent
 anchors.margins: Theme.spacingL
 spacing: 0

 Repeater {
 model: root.chars.length
```

Д

### Дед ИИ

08:56

Виджу причину. Виновата эта строка в обработчике ограничения:

```
if (text.length > maxLen) {
 inputField.text = text.substring(0, maxLen)
 return
}
```

Когда ты **редактируешь середину строки** и длина уже равна `maxLen` — этот блок не срабатывает, значит дело не здесь.

Курсор прыгает в конец из-за ``typingSession.setInputText(text)`` — в C++ `m_inputText` обновляется, и если где-то QML слушает `inputTextChanged` и синхронизирует `inputField.text` обратно — это сбрасывает курсор.

Покажи `Connections` на `typingSession`:

```
sed -n '70,105p' qml/components/TypingArea.qml
```

# Дед ИИ



Ответил: Claude

Заново

Озвучить

?



08:59

korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost\$ sed -n '70,105p' qml/components/TypingArea.qml

// — Сигналы сессии

---

```
Connections {
 target: typingSession

 function onInputTextChanged() {
 if (typingSession.ownerTag !== root.ownerTag) return
 root.cachedInputText = typingSession.inputText
 }

 function onCursorPosChanged() {
 if (typingSession.ownerTag !== root.ownerTag) return
 root.cachedCursorPos = typingSession.cursorPos
 }

 function onSessionFinished(result) {
 if (typingSession.ownerTag !== root.ownerTag) return
 inputField.text = ""
 inputField.enabled = false
 finishOverlay.visible = true
 }

 function onActiveChanged() {
 if (typingSession.ownerTag !== root.ownerTag) return
 if (!typingSession.active) {
 root.cachedInputText = ""
 root.cachedCursorPos = 0
 }
 }
}
```

// — Поле ввода

---

```
TextField {
 id: inputField
```

Д

**Дед ИИ**

08:59

Нашёл причину.

**ВИНОВНИК**

## Дед ИИ

Сам по себе этот обработчик **не трогает** `inputField.text` — он обновляет только `cachedInputText`. Но покажи как используется `cachedInputText` дальше в файле:

```
grep -n 'cachedInputText' qml/components/TypingArea.qml
```

Скорее всего где-то есть биндинг вида:

```
inputField.text: root.cachedInputText
```

или присвоение `inputField.text = root.cachedInputText` — вот оно и сбрасывает курсор в конец при каждом изменении.

 Ответил: Claude

 Ответил: Claude

 Ответил: Claude

 Заново

 Озвучить

?



09:00

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -n
'cachedInputText' qml/components/TypingArea.qml
17: property string cachedInputText: ""
31: root.cachedInputText = typingSession.inputText
47: root.cachedInputText = ""
63: root.cachedInputText = ""
78: root.cachedInputText = typingSession.inputText
96: root.cachedInputText = ""
169: property string typed: index < root.cachedInputText.length
170: ? root.cachedInputText[index] : "" // ← кэш
```

Д

Дед ИИ

09:00

`cachedInputText` нигде не пишется обратно в `inputField.text` — значит он не виновник.

Курсор прыгает в конец по другой причине. В `TextField` при программном изменении `text` (даже через `inputField.text = text.substring(0, maxLen)`) — Qt **автоматически** ставит курсор в конец. Это стандартное поведение `TextField`.

Но у тебя это срабатывает даже когда длина не превышает `maxLen` ... Покажи строки 25–40:

```
sed -n '25,42p' qml/components/TypingArea.qml
```

## Дед ИИ

текста до `onTextChanged`.

 Ответил: Claude

 Ответил: Claude

 Заново

 Озвучить

?



09:01

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ sed -n '25,42p'
qml/components/TypingArea.qml
// Не синхронизируем если сессия чужая
if (typingSession.ownerTag !== root.ownerTag) {
 console.log("[TypingArea] syncCache SKIP ownerTag=", root.ownerTag,
 "sessionOwner=", typingSession.ownerTag)
 return
}
root.cachedInputText = typingSession.inputText
root.cachedCursorPos = typingSession.cursorPos
console.log("[TypingArea] syncCache ownerTag=", root.ownerTag,
 "sessionOwner=", typingSession.ownerTag,
 "inputText=", typingSession.inputText.substring(0,20),
 "cursorPos=", typingSession.cursorPos)
}

function forceInputFocus() {
 syncCache()
 inputField.enabled = true
 inputField.forceActiveFocus()
}
```

Д

Дед ИИ

09:01

Покажи строки 1-25 — хочу увидеть начало файла и откуда  
вызывается `syncCache` :

```
sed -n '1,25p' qml/components/TypingArea.qml
```

 Ответил: Claude

 Ответил: Claude

 Заново

 Озвучить

?



09:01

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ sed -n '1,25p'
qml/components/TypingArea.qml
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts
import TypeBoost 1.0

ColumnLayout {
 id: root
 spacing: Theme.spacingM
```

## Дед ИИ

```
property string language: ""
property string layoutId: ""
property string ownerTag: ""

property string cachedInputText: ""
property int cachedCursorPos: 0

property var chars: currentText.split("")

// — Публичные функции
```

```
function syncCache() {
// Не синхронизируем если сессия чужая
```

Д

### Дед ИИ

09:02

Теперь картина полная. `syncCache` только читает из сессии в кэш, `inputField.text` нигде не перезаписывается при вводе.

### Реальная причина

Курсор прыгает в конец потому что в C++ `setInputText` делает `text.left(m_targetText.length())` и потом `emit inputTextChanged()`. Qt внутри `TextField` при получении нового значения через `WRITE` `setInputText` не знает позицию курсора — он просто получает новую строку и ставит курсор в конец.

Это происходит потому что `inputText` — это `Q_PROPERTY` с `WRITE`, и `TextField` привязан к нему через `text ↔ setInputText`. При каждом вызове `setInputText` Qt обновляет поле и сбрасывает курсор.

### Проверка

Подтверди: `inputField` нигде не имеет биндинга `text: typingSession.inputText`?

```
grep -n 'typingSession.inputText|text:' qml/components/TypingAr
```

Если такого биндинга нет — курсор не должен прыгать от `setInputText`. Тогда причина именно в блоке ограничения длины:

```
if (text.length > maxLen) {
 inputField.text = text.substring(0, maxLen) // ← это сбрас
 return
}
```



## Дед ИИ

символ, длина на him становится `maxLen + 1`, обрезает строку, и курсор летит в конец.

### Решение

Нужно после обрезки **восстанавливать позицию курсора**:

```
if (text.length > maxLen) {
 const cursorPos = inputField.cursorPosition // ← запомнить
 inputField.text = text.substring(0, maxLen)
 inputField.cursorPosition = Math.min(cursorPos - 1, maxLen)
 return
}
```



`cursorPos - 1` потому что пользователь вставил один лишний символ — после его удаления курсор должен стоять там же где был до вставки.

Попробуй и скажи — помогло ли.

Ответил: Claude

Заново

Озвучить

?



09:11

почти как надо – курсор теперь в конец не прыгает. но он остается на месте и вообще не двигается. то есть обычно курсор при вводе символа сдвигается вправо. сейчас же он остается на месте, то есть новые символы появляются не слева он курсора, а справа. может надо к позиции прибавлять 1? или это будет ошибкой в некоторых случаях?

Д

Дед ИИ

09:11

Да, надо прибавлять 1, но только если курсор не вышел за `maxLen` :

```
if (text.length > maxLen) {
 const cursorPos = inputField.cursorPosition
 inputField.text = text.substring(0, maxLen)
 inputField.cursorPosition = Math.min(cursorPos, maxLen) //
 return
}
```



Логика простая:

- курсор после вставки символа уже стоит **после** него — то есть `cursorPos` уже учитывает сдвиг вправо
- `- 1` было лишним — оно отматывало курсор назад
- `Math.min(..., maxLen)` оставляем — на случай если курсор оказался за границей строки

# Дед ИИ

?  09:14

все работает как надо

теперь вернемся к процентам

мне надо понять как считается статистика (средний процент ошибок)

Д **Дед ИИ** 09:14

Покажи как считается статистика:

```
cat src/core/StatisticsEngine.cpp
```

 Ответил: **Claude**

 Ответил: **Claude**

 **Заново**

 **Озвучить**

?  09:14

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ cat
src/core/StatisticsEngine.cpp
#include "StatisticsEngine.h"
#include "ProgressTracker.h"
#include "models/SessionResult.h"
```

```
#include <QMap>
```

```
StatisticsEngine::StatisticsEngine(ProgressTracker *tracker, QObject
*parent)
: QObject(parent), m_tracker(tracker)
{}

// ——— Helpers
```

```
static QMap buildSummary(const QList<SessionResult>
&sessions)
{
 QMap m;
```

```
 qreal totalSec = 0.0;
 qreal sumCpm = 0.0;
 qreal sumErr = 0.0;
 qreal bestCpm = 0.0;
```

```
 for (const auto &s : sessions) {
 totalSec += s.durationSec;
 sumCpm += s.cpm();
 sumErr += s.errorRate();
```

## Дед ИИ

```

const m = QMap<QString, double>();
m[QStringLiteral("totalSessions")] = n;
m[QStringLiteral("totalSeconds")] = totalSec;
m[QStringLiteral("totalMinutes")] = totalSec / 60.0;
m[QStringLiteral("avgCpm")] = n > 0 ? sumCpm / n : 0.0;
m[QStringLiteral("avgWpm")] = n > 0 ? (sumCpm / n) / 5.0 : 0.0;
m[QStringLiteral("avgErrorRate")] = n > 0 ? sumErr / n : 0.0;
m[QStringLiteral("bestCpm")] = bestCpm;
m[QStringLiteral("bestWpm")] = bestCpm / 5.0;
return m;
}

```

// — Public API

```

QVariantMap StatisticsEngine::summaryForRange(const QDateTime
&from,
const QDateTime &to) const
{
return buildSummary(m_tracker->sessionsInRange(from, to));
}

```

```

QVariantList StatisticsEngine::dailyDataForRange(const QDateTime
&from,
const QDateTime &to) const
{
const auto sessions = m_tracker->sessionsInRange(from, to);

```

```

// Group by local date string
QMap<QString, QList<SessionResult>> byDate;
for (const auto &s : sessions) {
const QString dateKey =
s.timestamp.toLocalTime().date().toString(Qt::ISODate);
byDate[dateKey].append(s);
}

```

```

QVariantList result;
for (auto it = byDate.cbegin(); it != byDate.cend(); ++it) {
const auto &list = it.value();
QVariantMap point;
point[QStringLiteral("date")] = it.key();

```

```

qreal totalSec = 0.0, sumCpm = 0.0, sumErr = 0.0;
for (const auto &s : list) {
totalSec += s.durationSec;
sumCpm += s.cpm();
sumErr += s.errorRate();
}
const int n = list.size();
point[QStringLiteral("minutes")] = totalSec / 60.0;
point[QStringLiteral("avgCpm")] = sumCpm / n;
point[QStringLiteral("avgWpm")] = (sumCpm / n) / 5.0;
point[QStringLiteral("avgErrorRate")] = sumErr / n;

```

## Дед ИИ

```
QVariantMap StatisticsEngine::summaryLastDays(int days) const
{
 const QDateTime to = QDateTime::currentDateTimeUtc();
 const QDateTime from = to.addDays(-days);
 return summaryForRange(from, to);
}
```

```
QVariantList StatisticsEngine::dailyDataLastDays(int days) const
{
 const QDateTime to = QDateTime::currentDateTimeUtc();
 const QDateTime from = to.addDays(-days);
 return dailyDataForRange(from, to);
}
```

Д

### Дед ИИ

09:15

#### Как считается средний процент ошибок

Логика простая — **среднеарифметическое** `errorRate` по всем сессиям:

```
sumErr += s.errorRate();
// ...
m["avgErrorRate"] = n > 0 ? sumErr / n : 0.0;
```

То есть берётся `errorRate` каждой сессии и делится на количество сессий.

#### Есть ли здесь проблема?

Зависит от того, что возвращает `s.errorRate()` в `SessionResult`. Покажи:

```
cat src/models/SessionResult.h
cat src/models/SessionResult.cpp
```

или если `.cpp` нет:

```
grep -n 'errorRate|errorCount|correctChars|targetText|totalChar
```

Суть вопроса: `SessionResult::errorRate()` считается так же как мы исправили в `TypingSession` — через `errors / targetText.length` — или по-старому через `errorCount / inputText.length`?



Ответил: Claude

## Дед ИИ

?

```

korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ cat
src/models/SessionResult.h
#pragma once

#include <QString>
#include <QDateTime>

/
* @brief Immutable result of one completed typing session (one text
passage).
*
* Stored as a JSON object inside progress.json by ProgressTracker.
*/
struct SessionResult
{
 QDateTime timestamp; ///< When the session ended (UTC)
 QString lessonId; ///< Lesson identifier, empty for free practice
 QString language; ///< "ru" | "en"
 QString layoutId; ///< e.g. "qwerty", "йцукен"

 int totalChars = 0; ///< Characters in the target text
 int correctChars = 0; ///< Correctly typed characters
 int errorCount = 0; ///< Cumulative incorrect key presses
 qreal durationSec = 0.0; ///< Elapsed seconds (start → last correct
char)

 / Characters per minute — only correct chars count. */
 qreal cpm() const
 {
 if (durationSec <= 0.0) return 0.0;
 return (correctChars / durationSec) * 60.0;
 }

 / Words per minute (CPM / 5, standard definition). */
 qreal wpm() const { return cpm() / 5.0; }

 / Error rate in percent [0, 100]. */
 qreal errorRate() const
 {
 if (totalChars == 0) return 0.0;
 return static_cast<qreal>(errorCount) / totalChars * 100.0;
 }
};
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -nE
'errorRate|errorCount|correctChars|targetText|totalChars'
src/models/SessionResult.h src/models/SessionResult.cpp 2> /dev/null
src/models/SessionResult.h:18: int totalChars = 0; ///< Characters in
the target text
src/models/SessionResult.h:19: int correctChars = 0; ///< Correctly
typed characters
src/models/SessionResult.h:20: int errorCount = 0; ///< Cumulative
incorrect key presses
src/models/SessionResult.h:27: return (correctChars / durationSec) *
60.0;

```

# Дед ИИ

Д

Дед ИИ

09:17

Вижу проблему

Формула в `SessionResult::errorRate()` :

```
return static_cast<qreal>(errorCount) / totalChars * 100.0;
```

Здесь `errorCount` — это **накопленный счётчик нажатий неверных клавиш** за всю сессию. То есть если пользователь нажал неверную клавишу 5 раз на одной и той же позиции — это посчитается как 5 ошибок.

При этом `totalChars` — длина эталонного текста, например 50 символов.

Итого: `errorRate` может легко превысить 100%, например  $60 / 50 * 100 = 120\%$ .

---

## Как должно быть

`errorCount` в `SessionResult` — это **не то же самое** что количество неверных позиций. Это сырой счётчик нажатий.

Есть два пути:

### Вариант 1 — использовать `correctChars`

Он уже есть в структуре и считается правильно:

```
qreal errorRate() const
{
 if (totalChars == 0) return 0.0;
 const int errors = totalChars - correctChars;
 return static_cast<qreal>(errors) / totalChars * 100.0;
}
```

Тогда `errorRate` всегда в  $[0, 100]$  и отражает долю **позиций**, где пользователь в итоге ошибся.

### Вариант 2 — оставить `errorCount` как есть

Тогда это метрика "сколько лишних нажатий сделал пользователь" — полезна сама по себе, но не как процент.

---

## Что я рекомендую

**Вариант 1.** Он согласуется с формулой которую мы утвердили в

## Дед ИИ

```
errorRate = wrongPositions / len(targetText) * 100
```

Но сначала проверь — как заполняется `correctChars` при сохранении сессии:

```
grep -n 'correctChars|totalChars|errorCount' src/core/TypingSes
```

Ответил: Claude

Ответил: Claude

Заново

Озвучить

?



09:19

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ grep -En
'correctChars|totalChars|errorCount' src/core/TypingSession.cpp
24:int TypingSession::errorCount() const { return m_errorCount; }
79: m_errorCount = 0;
152: ++m_errorCount;
213: result.totalChars = m_targetText.length();
214: result.correctChars = m_targetText.length() - m_errorCount; //
simplified
215: result.errorCount = m_errorCount;
```

Д

Дед ИИ

09:19

Вот и корень проблемы.

Что происходит сейчас

```
result.correctChars = m_targetText.length() - m_errorCount; //
```

`m_errorCount` — это накопленный счётчик всех неверных нажатий за сессию. Если пользователь 10 раз ошибся на одной позиции — `m_errorCount = 10`. Тогда:

```
correctChars = 50 - 10 = 40 // нормально
correctChars = 50 - 60 = -10 // уже абсурд
```

И `errorRate` в `SessionResult` считается через `errorCount / totalChars` — то есть тоже может быть больше 100%.

Правильное решение

## Дед ИИ

`correctChars`, просто надо использовать ту же логику при сохранении результата.

В `TypingSession.cpp` найди метод где заполняется `result` (строки 210–215) и замени:

```
// Считаем correctChars по финальному состоянию введённого текста
int correctChars = 0;
const int len = qMin(m_inputText.length(), m_targetText.length());
for (int i = 0; i < len; ++i) {
 if (m_inputText.at(i) == m_targetText.at(i))
 ++correctChars;
}

result.totalChars = m_targetText.length();
result.correctChars = correctChars;
result.errorCount = m_errorCount; // оставляем как есть — это
```

И в `SessionResult.h` исправь `errorRate()`:

```
qreal errorRate() const
{
 if (totalChars == 0) return 0.0;
 const int errors = totalChars - correctChars;
 return static_cast<qreal>(errors) / totalChars * 100.0;
}
```

### Итог

| | До | После |  
 |---|---|---|  
 | `correctChars` | `totalChars - errorCount` (может быть < 0) |  
 количество верных позиций в финальном тексте |  
 | `errorRate` в `SessionResult` | `errorCount / totalChars` (может быть > 100%) | `(totalChars - correctChars) / totalChars` — всегда `[0, 100]` |  
 | `errorCount` | используется везде | остаётся как сырая метрика, не влияет на проценты |



Ответил: Claude

Заново

Озвучить

?



09:22

разве не будет так, что в итоге процент ошибок всегда будет 0, ведь чтобы сессия закончилась, нужно ввести всю строку правильно?  
 ну то есть если я вводил вводил с ошибками, затем наконец их все исправил, то в финальной версии моего ответа 0 ошибок и сессия завершается.



# Дед ИИ

Д

ДЕД ИИ

09.22

Ты абсолютно прав. Смотри на `checkCompletion` :

```
if (m_inputText.length() < m_targetText.length()) return;
if (m_inputText != m_targetText) return; // ← сессия завершает
```

Сессия завершается **только когда введённый текст полностью совпадает с эталоном**. Значит в момент завершения `correctChars` всегда будет равен `totalChars` , и `errorRate` всегда будет 0%.

Значит `errorCount` — единственная полезная метрика

`m_errorCount` накапливает все неверные нажатия за сессию и **не обнуляется при исправлении**. Это и есть реальная мера того, насколько легко пользователю дался текст.

Правильная формула для `errorRate` тогда такая:

```
errorRate = errorCount / totalChars * 100
```

Но проблема в том что она может быть больше 100%. Это не страшно математически, но выглядит странно в UI.

**Компромисс — ограничить сверху**

```
qreal errorRate() const
{
 if (totalChars == 0) return 0.0;
 const qreal rate = static_cast<qreal>(errorCount) / totalCh
 return qMin(rate, 100.0); // ← не больше 100%
}
```

Это честно: если ошибок было больше чем символов — всё равно показываем 100%, что означает "очень плохо".

**Как тебе такой подход?**

Или хочешь другую шкалу — например логарифмическую, чтобы 200 ошибок на 50 символов не выглядело так же как 50 ошибок на 50?

# Дед ИИ

 Заново

 Озвучить

?



09:26

смотри, я хочу сделать так. наверное это будет и проще, и правильнее.

во время сессии мы не сбрасываем до нуля процент ошибок, если пользователь свою ошибку исправил. мы этот процент накапливаем. то есть если я ввел неверный символ, процент стал равен 5, затем я этот символ стер и ввел верный, процент не упал до нуля. далее если я опять совершил ошибку, процент должен прибавиться. при исправлении также не должен падать. и получится так, что в конце сессии вся строка будет введена верно, а вот процент ошибок будет говорить об истории ввода. и тогда средний процент в статистике будет опираться не на количество ошибок, а на процент ошибок в каждой конкретной сессии.

Д

**Дед ИИ**

09:27

Логика понятна и она правильная. Это называется **накопленный процент ошибок** — он отражает историю ввода, а не финальное состояние.

## Формула

```
errorRate = накопленных ошибок / totalChars * 100
```

Где **накопленных ошибок** — это количество позиций, где пользователь **хоть раз** ввёл неверный символ. То есть:

- ввёл неверно на позиции 3 → +1 ошибка
- стёр и ввёл верно на позиции 3 → ошибка **не убирается**
- ввёл неверно на позиции 3 снова → ошибка **не дублируется** (позиция уже помечена)

## Что нужно изменить в C++

Вместо счётчика `m_errorCount` (который сейчас считает все нажатия подряд) нужен **набор позиций где была ошибка**:

```
// TypingSession.h
QSet<int> m_errorPositions; // позиции где пользователь хоть р
```



## Дед ИИ

```
const int newPos = limited.length() - 1;
if (newPos < m_targetText.length()) {
 if (limited.at(newPos) != m_targetText.at(newPos)) {
 m_errorPositions.insert(newPos); // ← добавляем по
 }
}
}
```

В `updateStats` :

```
const int total = m_targetText.length();
m_errorRate = (total > 0)
 ? (static_cast<qreal>(m_errorPositions.size()) / total * 10
 : 0.0;
```

При сохранении результата:

```
result.totalChars = m_targetText.length();
result.correctChars = m_targetText.length() - m_errorPositions.
result.errorCount = m_errorPositions.size(); // теперь это у
```

При сбросе сессии добавить очистку:

```
m_errorPositions.clear();
```

### Что меняется в поведении

|                                           |                                      |                                 |                                                                                     |
|-------------------------------------------|--------------------------------------|---------------------------------|-------------------------------------------------------------------------------------|
| Ситуация                                  | До                                   | После                           |                                                                                     |
| --- --- ---                               |                                      |                                 |                                                                                     |
| Ввёл неверно, исправил                    | <code>errorRate</code> падает        | <code>errorRate</code> остаётся |                                                                                     |
| Ввёл неверно дважды на одной позиции      | <code>errorRate</code> растёт дважды |                                 |                                                                                     |
| <code>errorRate</code> растёт один раз    |                                      |                                 |                                                                                     |
| Финальный <code>errorRate</code> в сессии | всегда 0%                            | отражает историю                |                                                                                     |
| ввода                                     |                                      |                                 |                                                                                     |
| Диапазон                                  | <code>[0, ∞)</code>                  | <code>[0, 100]</code>           |  |

Утверждаешь — покажу точные строки для правки в файлах.

 Ответил: Claude

 Заново

 Озвучить

# Дед ИИ

Д

Дед ИИ

09:29

Покажи текущий полный `updateStats` и `reset / start` :

```
sed -n '160,230p' src/core/TypingSession.cpp
```

Чтобы написать точный код без угадывания структуры.



Ответил: Claude



Ответил: Claude



Ответил: Claude

Заново

Озвучить

?



09:30

```
korneploddviktor@LAPTOP-ANBC2JA3:~/TypeBoost$ sed -n '160,230p'
src/core/TypingSession.cpp
emit cursorPosChanged();
```

```
updateStats();
checkCompletion();
}
```

```
// — Private
```

```
void TypingSession::updateStats()
{
 if (!m_timerStarted) return;
 if (!m_timer.isValid()) return;

 const qreal elapsed = (m_pausedElapsed + m_timer.elapsed()) /
 1000.0; // seconds
 if (elapsed <= 0.0) return;

 // Count correct chars typed so far
 int correct = 0;
 const int len = qMin(m_inputText.length(), m_targetText.length());
 for (int i = 0; i < len; ++i) {
 if (m_inputText.at(i) == m_targetText.at(i))
 ++correct;
 }

 m_cpm = (correct / elapsed) * 60.0;

 int errors = 0;
 const int checked = m_inputText.length(); // только введённые
 позиции
 for (int i = 0; i < checked; ++i) {
```

## Дед ИИ

```

const int total = m_targetText.length();
m_errorRate = (total > 0) ? (static_cast<qreal>(errors) / total * 100.0) :
0.0;

emit statsUpdated();
}

void TypingSession::checkCompletion()
{
 if (m_inputText.length() < m_targetText.length()) return;
 if (m_inputText != m_targetText) return;

 // Session complete
 m_active = false;
 const qreal elapsed = m_timerStarted
? ((m_pausedElapsed + m_timer.elapsed()) / 1000.0)
: 0.0;

 SessionResult result;
 result.timestamp = QDateTime::currentDateTimeUtc();
 result.lessonId = m_lessonId;
 result.language = m_language;
 result.layoutId = m_layoutId;
 result.totalChars = m_targetText.length();
 result.correctChars = m_targetText.length() - m_errorCount; //
simplified
 result.errorCount = m_errorCount;
 result.durationSec = elapsed;

 if (m_tracker)
 m_tracker->record(result);

 emit sessionFinished(result);
 emit activeChanged();
}

```

Д

### Дед ИИ

09:30

Вот полный обновлённый `updateStats` и правки в `checkCompletion` :

В ``TypingSession.h`` добавь поле:

```
QSet<int> m_errorPositions;
```

И не забудь `#include <QSet>` вверху `.h` или `.cpp` .

В ``setInputText`` замени `++m_errorCount` на:

```

if (limited.length() > m_inputText.length()) {
 const int newPos = limited.length() - 1;
 if (newPos < m_targetText.length()) {
 if (limited.at(newPos) != m_targetText.at(newPos)) {
 m_errorPositions.insert(newPos); // ← вместо ++m_e
 }
 }
}

```

# Дед ИИ

``updateStats`` полностью:

```
void TypingSession::updateStats()
{
 if (!m_timerStarted) return;
 if (!m_timer.isValid()) return;

 const qreal elapsed = (m_pausedElapsed + m_timer.elapsed())
 if (elapsed <= 0.0) return;

 // Считаем верно введенные символы по текущему состоянию (д
 int correct = 0;
 const int len = qMin(m_inputText.length(), m_targetText.len
 for (int i = 0; i < len; ++i) {
 if (m_inputText.at(i) == m_targetText.at(i))
 ++correct;
 }

 m_cpm = (correct / elapsed) * 60.0;

 // Процент ошибок — накопленный, не падает при исправлении
 const int total = m_targetText.length();
 m_errorRate = (total > 0)
 ? (static_cast<qreal>(m_errorPositions.size()) / total
 : 0.0;

 emit statsUpdated();
}
```

``checkCompletion`` — исправить две строки:

```
result.totalChars = m_targetText.length();
result.correctChars = m_targetText.length() - m_errorPositions.
result.errorCount = m_errorPositions.size(); // уникальные п
result.durationSec = elapsed;
```

В ``reset`` (где `m_errorCount = 0`) добавь:

```
m_errorPositions.clear();
```

Покажи строки сброса чтобы убедиться:

```
sed -n '70,100p' src/core/TypingSession.cpp
```

# Дед ИИ



Ответил: Claude



Заново



Озвучить

?



09:32

где reset

Д

Дед ИИ

09:32

```
grep -n 'reset|clear|m_errorCount\s*=' src/core/TypingSession.c
```



Ответил: Claude



Ответил: Claude



Заново



Озвучить

?



09:52

Владимир Александрович, добрый день! Я Демаков Никита, группа 5130201/40001, пишу для согласования темы проекта для Цифровой аналитики.

Десктопное приложение клавиатурного тренажера.

Сервис актуален для людей, желающих повысить скорость своей печати на клавиатуре или научиться печатать "в слепую".

Краткое описание функционала.

На экране расположено поле ввода однострочного текста. Под этой строкой расположен текст, который нужно печатать. Каждый раз, когда пользователь напечатал весь текст, расположенный ниже поля ввода, содержимое поля ввода очищается, а текст ниже него обновляется. Таким образом, пользователь может бесконечно тренировать свою печать.

Для отслеживания собственного прогресса после каждого обновления строки в правом верхнем углу будет отображаться скорость печати (неошибочные символы в минуту) и процент ошибок (ошибочно введенных символов). Время для выявления скорости будет засекается в момент ввода первого символа (необязательно правильного) и останавливаться в момент правильного ввода последнего символа.

Ниже строки ввода и текста для печати отображается интерактивная раскладка клавиатуры. Она нужна для того, чтобы пользователь брал в привычку при печати смотреть не на клавиатуру, а на экран. Клавиатура на экране является интерактивной, поскольку она визуально подсказывает пользователю, какую клавишу нужно нажать в текущий момент (клавиша раскрашивается отличным от других цветом), а также для нее можно включать и выключать следующие опции: раскраска клавиш в соответствии с пальцами, графическая

## Дед ИИ

...иногда идея типична: правильное распределение пальцев рук по ней. Это означает, что каждому пальцу присваивается область клавиатуры, за которую он отвечает.

Более наглядное объяснение этого принципа представлено на рисунке. Описанная в предыдущем абзаце интерактивная клавиатура должна соответствовать распределению пальцев как на рисунке.

Используемые раскладки: "ЙЦУКЕН", "Русская (машинопись)", "Макинтош", "ЯВЕРТЫ"; "QWERTY", "Colemak", "Workman", "Dvorak".

Используемый стек: C++ / Qt

Добрый день, в целом нормально. Я бы еще добавил какую-то аналитику в части отслеживания собственного прогресса. Например, сколько минут (секунд) за последний день/неделю/месяц/настраиваемый интервал пользователь печатал. Как изменились параметры скорости и количества ошибок (средние) за этот период и т.п.

это моя с преподавателем переписка, в которой я ему рассказываю что хочу сделать. он также дает правки в само задание, то есть требует чтобы я добавил аналитику.

сформулируй на основе этой переписки и того что мы с тобой сделали полноценный текст технического задания. оформи все как фрагмент латех-документа

Д **Дед ИИ** 09:52  
Фрагмент LaTeX-документа — Техническое задание

```

\documentclass{article}
\usepackage{fontspec}
\setmainfont{TypeBoost}

\section{Техническое задание}

\subsection{Наименование и назначение}

Десктопное приложение \textbf{«TypeBoost»} --- клавиатурный тренажёр, предназначенный для повышения скорости и точности печати на клавиатуре, а также для формирования навыка слепой печати.

\subsection{Функциональные требования}

\subsubsection{Тренировочный режим}

\begin{itemize}
\item На экране отображается однострочное поле ввода и эталонный текст под ним, который необходимо воспроизвести.
\item Длина вводимого текста ограничена длиной эталонной строки
\end{itemize}

```



## Дед ИИ

по количеству итераций:

\item Отсчёт времени начинается в момент ввода первого символа (вне зависимости от его корректности) и останавливается в момент корректного ввода последнего символа эталонной строки.  
\end{itemize}

\subsubsection{Метрики в реальном времени}

В процессе тренировки в интерфейсе непрерывно отображаются:

\begin{itemize}  
\item \textbf{Скорость печати} --- количество верно введённых символов в минуту (CPM) и производная величина --- слов в минуту (WPM, из расчёта 5~символов на слово).  
\item \textbf{Процент ошибок} --- накопленная доля позиций, на которых пользователь хотя бы раз ввёл неверный символ, по отношению к длине эталонной строки:

$$E = (|P_{\text{error}}|)/(L_{\text{(target)}}) \times 100\%,$$

где  $P_{\text{error}}$  --- множество позиций с ошибкой,  
 $L_{\text{(target)}}$  --- длина эталонного текста.  
Процент ошибок не уменьшается при исправлении опечатки и всегда принадлежит диапазону  $[0, 100]$ .  
\end{itemize}

\subsubsection{Интерактивная экранная клавиатура}

\begin{itemize}  
\item Под полем ввода отображается графическая раскладка клавиатуры.  
\item Клавиша, которую необходимо нажать в текущий момент, выделяется цветом.  
\item Предусмотрены настраиваемые опции:  
\begin{itemize}  
\item раскраска клавиш по зонам пальцев рук;  
\item графическая подсказка о том, каким пальцем следует нажать текущую клавишу;  
\item возможность скрыть экранную клавиатуру полностью.  
\end{itemize}  
\item Распределение пальцев по зонам соответствует общепринятой методике слепой десятипальцевой печати.  
\end{itemize}

\subsubsection{Поддерживаемые раскладки}

\begin{table}[h]  
\centering  
\begin{tabular}{ll}  
\hline  
\textbf{Русские} & \textbf{Латинские} \end{tabular}

# Дед ИИ

Максимов & Voronin \

ЯВЕРТЫ & Dvorak \

\hline

\end{tabular}

\caption{Поддерживаемые раскладки клавиатуры}

\end{table}

\subsection{Аналитика и отслеживание прогресса}

\subsubsection{Сохранение результатов сессий}

По завершении каждой тренировочной сессии сохраняется следующий набор данных:

\begin{itemize}

09:52

\item дата и время окончания сессии (UTC);

\item идентификатор урока и язык;

\item используемая раскладка;

\item длина эталонного текста (\texttt{totalChars});

\item количество верно введённых символов (\texttt{correctChars});

\item количество позиций с ошибкой (\texttt{errorCount});

\item продолжительность сессии в секундах.

\end{itemize}

\subsubsection{Статистика за период}

Приложение предоставляет агрегированную статистику за выбранный

временной интервал (день / неделя / месяц / произвольный диапазон):

\begin{itemize}

\item суммарное время тренировок (в минутах);

\item среднее и максимальное значения CPM и WPM;

\item средний процент ошибок, вычисляемый как среднеарифметическое

накопленных процентов ошибок по отдельным сессиям:

[

\bar{E} = \frac{1}{n} \sum\_{i=1}^n E\_i;

]

\item динамика показателей в разбивке по дням (для графического отображения прогресса).

\end{itemize}

\subsection{Нефункциональные требования}

\begin{itemize}

\item \textbf{Платформа:} десктоп (Windows / Linux / macOS).

\item \textbf{Стек:} C++17, Qt~6, QML.

\item \textbf{Хранение данных:} локальный JSON-файл (\texttt{progress.json}).

\item \textbf{Интерфейс:} тёмная и светлая темы; моноширинный шрифт

в поле ввода и эталонном тексте.

Дед ИИ



Ответил: Claude



Заново



Озвучить

?



09:53

Актуальность

В повседневной жизни владельцы собак, посетители приютов и ветеринары часто сталкиваются с необходимостью быстро определить породу животного. Однако не каждый обладает достаточным опытом для точного визуального распознавания. Разработанный Telegram-бот позволяет любому пользователю сделать фотографию собаки и моментально получить результат, что особенно полезно для поиска потерявшихся животных, заполнения документов и ветеринарных или просто для удовлетворения любопытства. Простота использования через мессенджер делает технологию доступной даже для людей без специальной технической подготовки.

Цель и задачи

Целью данной работы является создание Telegram бота для визуального распознавания пород собак по фотографии с использованием двух последовательных моделей компьютерного зрения: детектора объектов YOLO для обнаружения собаки на изображении и классификатора EfficientNet-B4 для определения породы. Для достижения цели были решены следующие задачи:

- Обучение модели EfficientNet-B4 на датасете Stanford Dogs.
- Интеграция детектора объектов - загрузки предобученной модели YOLOv8, настройка детекции только класса "собака".
- Разработка бота:
  - реализация бота на библиотеке python-telegram-bot
  - добавление главного меню с кнопками: «Загрузить фотографию» - запрос изображения; «О боте» - информация о модели и точности.
  - реализация обработки изображений (JPEG, PNG, до 15 MB).
- Обреза области с каждой собакой.
- Классификация обрезанного изображения с помощью EfficientNet-B4.
- Вывод топ-3 пород с процентом уверенности.
- Отрисовка bounding box на исходном изображении и отправка результата пользователю.
- Поддержка сценария с несколькими собаками.
- Обработка ошибок и безопасности: проверка наличия собаки на фото, обработка опечаток клавиш, автоматическая очистка временной памяти после обработки.
- Оформление отчётов, документация

нет, сделай это в формате, как на фото

Результаты

09:53

Разработанный Telegram-бот позволяет пользователю загружать фотографию собаки, после чего автоматически обнаруживает животное на изображении с помощью YOLO, определяет породу с точностью 85-90% с использованием EfficientNet-B4 и выдаёт топ-3 наиболее вероятных породы с процентами уверенности, сопровождая результат визуализацией в виде обведённого прямоугольника.

Программа может быть использована в повседневной жизни владельцами собак, кинологами, ветеринарами и любителями животных для быстрой идентификации породы по фотографии, а также в образовательных целях для демонстрации работы современных систем компьютерного зрения и последовательного применения детекции и классификации.

Д

Дед ИИ

09:53



Редактирование картинок пока не поддерживается

Я могу описать что на фото — отправьте без подписи или спросите «что на этом фото». Подключим редактирование позже.

?



09:54

Актуальность

В повседневной жизни владельцы собак, посетители приютов и ветеринары часто сталкиваются с необходимостью быстро определить породу животного. Однако не каждый обладает достаточным опытом для точного визуального распознавания. Разработанный Telegram-бот позволяет любому пользователю сделать фотографию собаки и моментально получить результат, что особенно полезно для поиска потерявшихся животных, заполнения документов и ветеринарных или просто для удовлетворения любопытства. Простота использования через мессенджер делает технологию доступной даже для людей без специальной технической подготовки.

Цель и задачи

Целью данной работы является создание Telegram бота для визуального распознавания пород собак по фотографии с использованием двух последовательных моделей компьютерного зрения: детектора объектов YOLO для обнаружения собаки на изображении и классификатора EfficientNet-B4 для определения породы. Для достижения цели были решены следующие задачи:

- Обучение модели EfficientNet-B4 на датасете Stanford Dogs.
- Интеграция детектора объектов - загрузки предобученной модели YOLOv8, настройка детекции только класса "собака".
- Разработка бота:
  - реализация бота на библиотеке python-telegram-bot
  - добавление главного меню с кнопками: «Загрузить фотографию» - запрос изображения; «О боте» - информация о модели и точности.
  - реализация обработки изображений (JPEG, PNG, до 15 MB).
- Обреза области с каждой собакой.
- Классификация обрезанного изображения с помощью EfficientNet-B4.
- Вывод топ-3 пород с процентом уверенности.
- Отрисовка bounding box на исходном изображении и отправка результата пользователю.
- Поддержка сценария с несколькими собаками.
- Обработка ошибок и безопасности: проверка наличия собаки на фото, обработка опечаток клавиш, автоматическая очистка временной памяти после обработки.
- Оформление отчётов, документация

нет, сделай это в формате как в примере на фото

## Дед ИИ

Я могу **описать** что на фото — отправьте без подписи или спросите «что на этом фото». Подключим редактирование позже.

?  09:55

нет, сделай это в таком формате

актуальность  
цели и задачи  
результаты работы

Д **Дед ИИ** 09:55

Покажи пример оформления из твоего шаблона или с прошлых работ — чтобы точно попасть в стиль и структуру разделов. Или скинь как выглядит шаблон `.tex` файла.

 Заново

 Озвучить

?  09:55

могу скинуть пдф?

Д **Дед ИИ** 09:55

Да, скидывай — я смогу его прочитать и подстроиться под нужный стиль оформления.

 Заново

 Озвучить

?  09:56

**ЦА.pdf**

114.1 KB

титульник писать не надо. надо сделать фрагмент с актуальностью, целями и задачами, результатами

Д **Дед ИИ** 09:57

````latex`

`\section*{Введение}`

`\subsection*{Актуальность}`

Владение навыком слепой десятипальцевой печати является одной из ключевых компетенций для специалистов, чья профессиональная деятельность связана с компьютером. Тем не менее большинство пользователей печатают неоптимальным способом, глядя на клавиатуру и задействуя лишь часть пальцев, что существенно снижает скорость и увеличивает количество ошибок.

Существующие веб-сервисы для тренировки печати либо не поддерживают русскоязычные раскладки в полном объёме, либо не предоставляют

Дед ИИ

сетью десктопное приложение лишено этих недостатков: оно работает полностью локально, поддерживает как русские, так и латинские раскладки, и сохраняет историю тренировок для последующего анализа.

\subsection*{Цель и задачи}

Целью данной работы является разработка десктопного приложения клавиатурного тренажёра \textbf{TypeBoost}, позволяющего пользователю тренировать скорость и точность печати, отслеживать собственный прогресс и формировать навык слепой десятипальцевой печати.

Для достижения поставленной цели решены следующие задачи:

\begin{itemize}

\item Реализация тренировочного режима с однострочным полем ввода, эталонным текстом и автоматической сменой строки после её корректного ввода.

\item Реализация физического ограничения длины ввода~--- невозможность

ввести больше символов, чем содержится в эталонной строке.

\item Разработка системы подсчёта метрик в реальном времени: скорость печати (CPM/WPM) и накопленный процент ошибок, не уменьшающийся при исправлении опечатки.

\item Реализация интерактивной экранной клавиатуры с подсветкой текущей клавиши, раскраской зон по пальцам и графической подсказкой правильного пальца.

\item Поддержка восьми раскладок клавиатуры: четырёх русских (ЙЦУКЕН, Русская машинпись, Макинтош, ЯБЕРТЫ) и четырёх латинских (QWERTY, Colemak, Workman, Dvorak).

\item Разработка модуля сохранения результатов сессий в локальный

JSON-файл с фиксацией времени, раскладки, количества верно введённых символов, уникальных позиций с ошибкой и продолжительности сессии.

\item Разработка модуля аналитики и визуализации прогресса: агрегированная статистика за произвольный временной интервал (день\,/\,неделя\,/\,месяц) с отображением суммарного времени тренировок, среднего и максимального CPM/WPM, среднего процента

ошибок и динамики показателей по дням.

\item Оформлена отчётная документация.

\end{itemize}

\subsection*{Результаты}

Разработано десктопное приложение \textbf{TypeBoost} на стеке C++17 / Qt~6 / QML, функционирующее полностью локально без подключения к сети.

Дед ИИ

\item тренировочный режим с автоматической сменой эталонной строки и физическим ограничением длины ввода;

\item отображение скорости печати (CPM и WPM) и накопленного процента ошибок в реальном времени; процент ошибок вычисляется как доля уникальных позиций с ошибкой от длины эталонного текста и не обнуляется при исправлении опечатки, что объективно отражает историю ввода;

\item интерактивную экранную клавиатуру с подсветкой текущей клавиши, цветовой разметкой зон по пальцам и графической подсказкой правильного пальца;

\item поддержку восьми раскладок (четыре русские и четыре латинские);

\item сохранение результатов каждой сессии в локальный файл \texttt{progress.json} и предоставление агрегированной аналитики за выбранный период~--- суммарное время тренировок, средние и максимальные показатели скорости, средний процент ошибок и графики динамики по дням.

\end{itemize}

Приложение может быть использовано студентами, офисными работниками и всеми, кто стремится повысить скорость и точность печати, сформировать навык слепой десятипальцевой печати и объективно отслеживать собственный прогресс.

...

 Заново

 Озвучить

?



10:20

реализуй раздел техническое задание по примеру из этого же файла

Д

Дед ИИ

10:20

Смотрю на структуру раздела «Техническое задание» из вашего PDF-файла. Вот фрагмент:

```
```\latex
\section{Техническое задание}

\subsection{Наименование программы}
```

Десктопное приложение клавиатурного тренажёра  
\textbf{«TypeBoost»}.

```
\subsection{Область применения}
```

Приложение предназначено для персональных компьютеров под управлением операционных систем Windows, Linux и macOS. Целевая аудитория~---

# Дед ИИ

\subsection{Назначение программы}

Приложение обеспечивает интерактивную тренировку печати: пользователь воспроизводит предложенный эталонный текст, получает обратную связь в реальном времени (скорость, процент ошибок, подсветка клавиш), а результаты каждой сессии сохраняются для последующего анализа прогресса.

\subsection{Требования к программе}

\subsubsection{Требования к функциональным характеристикам}

Программа должна обеспечивать:

\begin{enumerate}  
\item \textbf{Тренировочный режим.}  
\begin{itemize}  
\item Отображение однострочного поля ввода и эталонного текста под ним.  
\item Ограничение длины ввода длиной эталонной строки---- ввод лишних символов физически невозможен.  
\item Автоматическую очистку поля ввода и смену эталонного текста после корректного ввода всей строки.  
\item Подсветку уже введенных символов: верно введенные выделяются одним цветом, ошибочные---- другим.  
\end{itemize}  
\item \textbf{Отсчет времени.}  
\begin{itemize}  
\item Таймер запускается в момент ввода первого символа (вне зависимости от его корректности).  
\item Таймер останавливается в момент корректного ввода последнего символа эталонной строки.  
\end{itemize}  
\item \textbf{Метрики в реальном времени.}  
\begin{itemize}  
\item Скорость печати в символах в минуту (CPM) и словах в минуту (WPM, из расчёта 5~символов на слово); учитываются только верно введенные символы.  
\item Накопленный процент ошибок---- доля позиций эталонной строки, на которых пользователь хотя бы раз ввёл неверный символ:  
\begin{equation}  
E = \frac{|\setminus P\_{\setminus \text{error}}|}{|\setminus L\_{\setminus \text{target}}|} \times 100\%,  
\end{equation}  
где  $\setminus P_{\setminus \text{error}}$ ---- множество позиций с ошибкой,  $\setminus L_{\setminus \text{target}}$ ---- длина эталонного текста.  
Процент ошибок не уменьшается при исправлении опечатки

## Дед ИИ

```
\begin{itemize}
\item Отображение графической раскладки клавиатуры под полем ввода.
\item Подсветка клавиши, которую необходимо нажать в текущий момент.
\item Раскраска клавиш по зонам пальцев рук (настраиваемая опция).
\item Графическая подсказка о том, каким пальцем следует нажать текущую клавишу (настраиваемая опция).
\item Возможность полностью скрыть экранную клавиатуру (настраиваемая опция).
\item Распределение пальцев по зонам соответствует общепринятой методике слепой десятипальцевой печати.
\end{itemize}
```

```
\item \textbf{Поддержка раскладок клавиатуры.}
```

```
\begin{table}[h]
\centering
\begin{tabular}{ll}
\hline
\textbf{Русские раскладки} & \textbf{Латинские раскладки} \\
\hline
ЙЦУКЕН & QWERTY \\
Русская (машинопись) & Colemak \\
Макинтош & Workman \\
ЯВЕРТЫ & Dvorak \\
\hline
\end{tabular}
\caption{Поддерживаемые раскладки клавиатуры}
\end{table}
```

```
\item \textbf{Сохранение результатов сессий.}
```

По завершении каждой сессии в локальный файл

```
\texttt{progress.json}
```

сохраняются:

```
\begin{itemize}
\item дата и время окончания сессии (UTC);
\item идентификатор урока и язык тренировки;
\item используемая раскладка;
\item длина эталонного текста;
\item количество верно введенных символов;
\item количество уникальных позиций с ошибкой;
\item продолжительность сессии в секундах.
\end{itemize}
```

```
\item \textbf{Аналитика и отслеживание прогресса.}
```

Приложение предоставляет агрегированную статистику за выбранный

временной интервал (день\,/\.неделя\,/\.месяц\,/\.произвольный диапазон):

```
\begin{itemize}
```



# Дед ИИ

наконец-то

процентов ошибок по отдельным сессиям:

$$\bar{E} = \frac{1}{n} \sum_{i=1}^n E_i;$$

динамика показателей в разбивке по дням для графического отображения прогресса.

Требования к надёжности

Приложение не должно допускать потери сохранённых результатов

сессий при нештатном завершении работы.

Все пользовательские данные хранятся локально; подключение к сети не требуется.

Требования к составу и параметрам технических средств

Операционная система: Windows~10/11, Ubuntu~20.04 и выше,  
macOS~12 и выше.

Оперативная память: не менее 256~МБ.

Место на диске: не менее 100~МБ.

Наличие физической или экранной клавиатуры.

Требования к информационной и программной совместимости

10:21

Язык реализации: C++17.

Фреймворк: Qt~6 (модули QtCore, QtQuick, QtQuick.Controls, QtQuick.Layouts).

Язык описания интерфейса: QML.

Хранение данных: локальный файл в формате JSON.

Сборочная система: CMake~3.16 и выше.

Требования к программной документации

Программная документация должна включать:

настоящее техническое задание;

описание архитектуры приложения;

руководство пользователя.

Стадии и этапы разработки

# Дед ИИ

```
\table
\textbf{№} & \textbf{Этап} & \textbf{Результат} \\
\hline
1 & Анализ требований и проектирование архитектуры
& Схема архитектуры \\
2 & Реализация ядра (сессия, метрики, хранение данных)
& Рабочий C++ backend \\
3 & Разработка интерфейса (QML)
& Рабочий UI \\
4 & Реализация экранной клавиатуры
& Интерактивная клавиатура \\
5 & Реализация модуля аналитики
& Экран статистики \\
6 & Тестирование и отладка
& Стабильное приложение \\
7 & Оформление документации
& Отчёт \\
\hline
\end{tabular}
\caption{Этапы разработки}
\end{table}
```

\subsection{Порядок контроля и приёмки}

Приложение считается принятым при выполнении следующих условий:

```
\begin{itemize}
\item все функциональные требования реализованы и проверены вручную;
\item результаты сессий корректно сохраняются и отображаются в модуле аналитики;
\item приложение запускается и работает стабильно на целевой платформе.
\end{itemize}
```

````

 Ответил: **Claude**

 [Заново](#)

 [Озвучить](#)